INDEX

D26IT117 – Object Oriented Programming with Java

Practical	Topics	Date	Page No.
1 (A)	Vending Machine; Toll Booth	07-07-26	1
1 (B)	MiniBank Menu, BankInfo Record and Account Setup	07-07-26	4
2 (A)	Smart Thermostat; Cinema Show	13-07-26	8
2 (B)	Customer and Account Classes; Deposits and Withdrawals	13-07-26	11
3 (A)	Distinct Points; Duplicate Card Check	20-07-26	19
3 (B)	Account Equality and Formatting; Nested Address; Cloning	20-07-26	24
4 (A)	Password Strength; Chat-Log Filter	29-07-26	30
4 (B)	Input Validation; Command Parsing; Statement Formatting	29-07-26	34
5 (A)	Shape Areas; Payroll	03-08-26	36
5 (B)	Abstract Account and Account Types; Polymorphism	03-08-26	41
6 (A)	Remote Control Interfaces; Notification Senders	17-08-26	44
6 (B)	Banking Interfaces; Withdrawal Rules; Packages and Runnable JAR	17-08-26	48
7 (A)	Mini Test Runner; Form Validator	27-10-26	50
7 (B)	Custom Annotations and Reflective Account Validation	27-10-26	52
8 (A)	Guarded Calculator; Stock Issue	16-09-26	54
8 (B)	Custom Exceptions; Withdrawal and Transfer Handling	16-09-26	58
9 (A)	Counter Race; Seat Booking Race	16-09-26	62
9 (B)	Concurrent Account Workers and Synchronized Transactions	16-09-26	67
10 (A)	Thread-Pool Runner; Producer-Consumer	05-10-26	70

10 (B)	Transaction Processing, Queues and Deadlock	05-10-26	75
11 (A)	Save/Load Serialization; Log Analyzer	27-10-26	78
11 (B)	Account Persistence, Transaction Logs and Reports	27-10-26	82
12 (A)	Word Frequency; Unique Visitors	27-10-26	89
12 (B)	AccountStore, Sorting, Comparable and Concurrent Map	27-10-26	92
13 (A)	Generic Containers; Bounded Statistics	27-10-26	99
13 (B)	Generic Repository, Reflection and saveAll	27-10-26	101
14 (A)	Layered Application and JUnit Tests	27-10-26	105
14 (B)	Integrated MiniBank Application, Tests and Runnable JAR	27-10-26	105

PRACTICAL - 1

Aim: To develop a Java program to implement a vending machine using enum and switch expression to calculate the total amount paid and the change to be returned.
Code Screenshot:

PART - A

- **Vending_Machine:**

vending_machine.java 1

vending_machine.java > Java > vending_machine

```java
import java.util.Scanner;

public class vending_machine {

    enum Coin {
        ONE, TWO, FIVE, TEN
    }

    Run | Debug | Run main | Debug main
    public static void main(String[] args) {

        final int price = 15;
        int total = 0;

        Scanner scanner = new Scanner(System.in);

        while (total < price) {

            System.out.print(s: "Enter coin (ONE, TWO, FIVE, TEN): ");

            Coin selectedCoin = Coin.valueOf(
                scanner.nextLine().toUpperCase()
            );

            int coinValue = switch (selectedCoin) {
                case ONE -> 1;
                case TWO -> 2;
                case FIVE -> 5;
                case TEN -> 10;
            };

            total += coinValue;

            System.out.println("Total So Far: " + total);
        }

        System.out.println("Paid. Change: " + (total - price));

        scanner.close();
    }
}
```

- **Toll_booth:**



```java
import java.util.Scanner;

public class toll_booth {

    record Vehicle(String number, String type) {}

    Run | Debug | Run main | Debug main
    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        int totalToll = 0;
        int bikeCount = 0;
        int carCount = 0;
        int truckCount = 0;

        while (true) {

            System.out.print(s: "Enter vehicle number (or done): ");
            String number = scanner.nextLine();

            if (number.equalsIgnoreCase(anotherString: "done")) {
                break;
            }

            System.out.print(s: "Enter vehicle type (bike/car/truck): ");
            String type = scanner.nextLine().toLowerCase();

            Vehicle vehicle = new Vehicle(number, type);

            int toll = switch (vehicle.type()) {
                case "bike" -> 20;
                case "car" -> 50;
                case "truck" -> 150;
                default -> 0;
            };

            totalToll += toll;

            switch (vehicle.type()) {
                case "bike" -> bikeCount++;
                case "car" -> carCount++;
                case "truck" -> truckCount++;
            }
        }

        String mostFrequent;

        if (carCount >= bikeCount && carCount >= truckCount) {
            mostFrequent = "car";
        } else if (bikeCount >= carCount && bikeCount >= truckCount) {
            mostFrequent = "bike";
        } else {
            mostFrequent = "truck";
        }

        System.out.println("Total toll: " + totalToll);
        System.out.println("Most frequent: " + mostFrequent);

        scanner.close();
    }
}
```

Output Screenshot:

- **Vending_Machine:**

```
PS D:\CSPIT\OOP\Practical_1\Vending_Machine> java .\vending_machine.java
Enter coin (ONE, TWO, FIVE, TEN): one
Total So Far: 1
Enter coin (ONE, TWO, FIVE, TEN): five
Total So Far: 6
Enter coin (ONE, TWO, FIVE, TEN): two
Total So Far: 8
Enter coin (ONE, TWO, FIVE, TEN): ten
Total So Far: 18
Paid. Change: 3
PS D:\CSPIT\OOP\Practical_1\Vending_Machine>
```

- **Toll_booth:**

```
PS D:\CSPIT\OOP\Practical_1\Toll_Booth> java .\toll_booth.java
Enter vehicle number (or done): 1
Enter vehicle type (bike/car/truck): bike
Enter vehicle number (or done): 2
Enter vehicle type (bike/car/truck): truck
Enter vehicle number (or done): 3
Enter vehicle type (bike/car/truck): car
Enter vehicle number (or done): 4
Enter vehicle type (bike/car/truck): truck
Enter vehicle number (or done): 5
Enter vehicle type (bike/car/truck): truck
Enter vehicle number (or done): 6
Enter vehicle type (bike/car/truck): bike
Enter vehicle number (or done): done
Total toll: 540
Most frequent: truck
PS D:\CSPIT\OOP\Practical_1\Toll_Booth>
```

PART - B

- **MiniBank.java**

```
public class MiniBank {


    private static final Map<String, Account> accounts = new LinkedHashMap<>();

    private static final BankInfo BANK = new BankInfo("MiniBank", "CSPIT Branch");


    public static void main(String[] args) {

        if (Arrays.asList(args).contains("--demo"))

            { runDemo();

            return;

        }

        interactive();

    }


    private static void interactive()

        { System.out.println(BANK);

        try (Scanner scanner = new Scanner(System.in))

            { boolean running = true;

            while (running) {

                System.out.println("\n1.Open 2.Deposit 3.Withdraw 4.Transfer 5.Working hours 6.Exit");

                System.out.print("Choice: ");

                String input = scanner.nextLine().trim();

                try {

                    int choice = Integer.parseInt(input);

                    String result = switch (choice) {
```

```
                case 1 ->
                    open(scanner);
                case 2 ->
                    deposit(scanner);
                case 3 ->
                    withdraw(scanner);
                case 4 ->
                    transfer(scanner);
                case 5 ->
                    "Working hours: Monday-Saturday, 10:00 to 16:00.";
                case 6 -> {
                    running = false;
                    yield "Goodbye from MiniBank.";
                }
                default ->
                    "Invalid menu number. Please choose 1-6.";
            };
            System.out.println(result);
        } catch (NumberFormatException e)
            { System.out.println("Enter a numbered menu option.");
        }
    }
  }
}
```

- **Output :-**

```
PS D:\CSPIT\Fenil\OOP\Mini_Bank> javac -d out (Get-ChildItem -Recurs
>> java -cp out MiniBank
BankInfo[name=MiniBank, branch=CSPIT Branch]

1.Open 2.Deposit 3.Withdraw 4.Transfer 5.Working hours 6.Exit
Choice: 9
Invalid menu number. Please choose 1-6.

1.Open 2.Deposit 3.Withdraw 4.Transfer 5.Working hours 6.Exit
Choice: ABC
Enter a numbered menu option.
```

Learning Outcome:

The learning outcome of this practical is to understand and apply basic Java programming concepts such as enums, records, switch expressions, loops, user input, counters, and accumulators. Students will learn how to process user input, perform decision-making using switch expressions, calculate totals, track different categories, and determine the most frequent category.

PRACTICAL - 2

Aim: To develop a Java program to implement a toll booth using record and switch expression to calculate the total toll collected and determine the most frequently used vehicle type.

Code Screenshot:

- **cinema_show:**



```java
public class CinemaShow {

    private String title;
    private int seatsAvailable;
    private final int capacity;
    private static int totalBooked = 0;

    public CinemaShow(String title, int capacity) {
        this.title = title;
        this.capacity = capacity;
        this.seatsAvailable = capacity;
    }

    public CinemaShow(String title) {
        this(title, capacity: 100);
    }

    public boolean book(int n) {
        if (n <= seatsAvailable) {
            seatsAvailable -= n;
            totalBooked += n;
            return true;
        }
        return false;
    }

    public void cancel(int n) {
        seatsAvailable += n;
        if (seatsAvailable > capacity) {
            seatsAvailable = capacity;
        }
    }

    public int getSeatsAvailable() {
        return seatsAvailable;
    }

    public static int getTotalBooked() {
        return totalBooked;
    }

    Run | Debug | Run main | Debug main
    public static void main(String[] args) {

        CinemaShow show = new CinemaShow(title: "Avengers", capacity: 50);

        System.out.println("Book 20 seats: " + show.book(n: 20));
        System.out.println("Seats Available: " + show.getSeatsAvailable());

        System.out.println("Book 15 seats: " + show.book(n: 15));
        System.out.println("Seats Available: " + show.getSeatsAvailable());

        System.out.println(x: "Cancel 5 seats");
        show.cancel(n: 5);
        System.out.println("Seats Available: " + show.getSeatsAvailable());

        System.out.println("Book 25 seats: " + show.book(n: 25));
        System.out.println("Seats Available: " + show.getSeatsAvailable());

        System.out.println("Total Booked: " + CinemaShow.getTotalBooked());
    }
}
```

- **smart_thermostat:**



```java
class Thermostat{
    private String location;
    private int temperature;

    private static final int MIN = 16;
    private static final int MAX = 30;

    private static int activeCount = 0;

    public Thermostat(String location, int startTemp){
        this.location = location;

        if (startTemp >= MIN && startTemp <= MAX){
            this.temperature = startTemp;
        }
        else{
            this.temperature = 22;
        }
        activeCount++;
    }

    public Thermostat(String location){
        this(location,startTemp: 22);
    }

    public void raise(){
        if (temperature < MAX){
            temperature++;
        }
        else{
            System.out.println(x: "Already At Maximum (30)");
        }
    }

    public void lower(){
        if (temperature > MIN){
            temperature--;
        }
        else{
            System.out.println(x: "Already At Minimum (16)");
        }
    }

    public int getTemperature(){
        return  temperature;
    }

    public static int getActiveCount(){
        return activeCount;
    }
    Run | Debug | Run main | Debug main
    public static void main(String args[]){
        Thermostat c1 = new Thermostat(location: "Bedroom");
        Thermostat c2 = new Thermostat(location: "Hall",startTemp: 20);
        System.out.println("Initial Temperature: " + c2.getTemperature());
        System.out.println(x: "Raising Temperature....");
        for (int i = 0; i < 10; i++){
            c2.raise();
            System.out.println("Temperature: " + c2.getTemperature());
        }
        System.out.println(x: "Lowering Temperature....");
        for (int i = 0; i < 20; i++){
            c2.lower();
            System.out.println("Temperature: " + c2.getTemperature());
        }
        System.out.println("\nActive Thermostat :" + Thermostat.getActiveCount());
    }
}
```

Output Screenshot:

- **cinema_show:**

```
PS D:\CSPIT\OOP\Practical_2\cinema_show> java .\CinemaShow.java
Book 20 seats: true
Seats Available: 30
Book 15 seats: true
Seats Available: 15
Cancel 5 seats
Seats Available: 20
Book 25 seats: false
Seats Available: 20
Total Booked: 35
PS D:\CSPIT\OOP\Practical_2\cinema_show>
```

- **smart_thermostat:**

```
PS D:\CSPIT\OOP\Practical_2\smart_thermostat> java .\Thermostat.java
Initial Temperature: 20
Raising Temperature....
Temperature: 21
Temperature: 22
Temperature: 23
Temperature: 24
Temperature: 25
Temperature: 26
Temperature: 27
Temperature: 28
Temperature: 29
Temperature: 30
Lowering Temperature....
Temperature: 29
Temperature: 28
Temperature: 27
Temperature: 26
Temperature: 25
Temperature: 24
Temperature: 23
Temperature: 22
Temperature: 21
Temperature: 20
Temperature: 19
Temperature: 18
Temperature: 17
Temperature: 16
Already At Minimum (16)
Temperature: 16
Already At Minimum (16)
Temperature: 16
Already At Minimum (16)
Temperature: 16
Already At Minimum (16)
Temperature: 16
Already At Minimum (16)
Temperature: 16
Already At Minimum (16)
Temperature: 16

Active Thermostat :2
PS D:\CSPIT\OOP\Practical_2\smart_thermostat>
```

PART - B

- **Account.java**

```
package model;


import exception.*;
import java.io.*;
import java.util.Objects;
import model.annotation.Id;
import model.annotation.Positive;


public abstract class Account implements Transactable, Serializable {


    private static final long serialVersionUID = 1L;
    private static long counter = 1;
    @Id
    private final String accountNumber;
    private final String ownerName;
    @Positive
    private long balance;
    private boolean active = true;
    private transient Object lock = new Object();


    protected Account(String ownerName, long openingBalance) {
        if (ownerName == null || ownerName.isBlank() || openingBalance < 0) {
            throw new IllegalArgumentException("Owner and non-negative opening balance are required");
```

```
    }this.accountNumber = nextNumber();
    this.ownerName = ownerName;
    this.balance = openingBalance;
}

private static synchronized String nextNumber()
    { return String.format("AC%04d", counter++);
}

public final String getAccountNumber()
    { return accountNumber;
}

public final String getOwnerName()
    { return ownerName;
}

public final long getBalance()
    { synchronized (lock)
        { return balance;
    }
}

public final boolean isActive() {
```

```
        return active;
    }

    protected final Object accountLock()
        { return lock;
    }

    protected final long balanceUnsafe()
        { return balance;
    }

    @Override
    public final void deposit(long amount) throws InvalidAmountException
        { synchronized (lock)
            { validateAmount(amount);
            balance = Math.addExact(balance, amount);
        }
    }

    @Override
    public final boolean withdraw(long amount) throws InsufficientFundsException,
InvalidAmountException {
        synchronized (lock)
            { validateAmount(amount);
            if (!canWithdraw(amount)) {
```

```
            throw new InsufficientFundsException("Withdrawal denied by account policy", Math.max(0,
amount - Math.max(0, balance)));


        }balance -= amount;

        return true;

    }

}


public final void transfer(Account to, long amount) throws BankException

    { if (to == null) {

        throw new AccountNotFoundException("Destination account is required");


    }if (to == this) {

        throw new BankException("Cannot transfer to the same account");


    }Account first = this.accountNumber.compareTo(to.accountNumber) < 0 ? this : to;

    Account second = first == this ? to : this;

    synchronized (first.lock)

        { synchronized (second.lock)

            { try {

                validateAmount(amount);

                if (!canWithdraw(amount)) {

                    throw new InsufficientFundsException("Transfer denied by account policy", Math.max(0,
amount - Math.max(0, balance)));


                }this.balance -= amount;
```

```
                try {
                    to.balance = Math.addExact(to.balance, amount);
                } catch (ArithmeticException e)
                    { this.balance += amount;
                    throw new BankException("Transfer rolled back", e);
                }
            } finally {
                /* locks release automatically */ }
        }
    }
}

    private static void validateAmount(long amount) throws InvalidAmountException
    { if (amount <= 0) {
        throw new InvalidAmountException("Amount must be positive");

    }}

    public abstract double interestRate();

    public abstract boolean canWithdraw(long amount);

    public final double monthlyInterest() {
    return getBalance() * interestRate() / 1200.0;
    }
```

```
    @Override
    public final boolean equals(Object other) {
        return other instanceof Account a && accountNumber.equals(a.accountNumber);
    }

    @Override
    public final int hashCode() {
        return Objects.hash(accountNumber);
    }

    @Override
    public String toString() {
        return "%s{number=%s, owner=%s, balance=%d,
active=%s}".formatted(getClass().getSimpleName(), accountNumber, ownerName, getBalance(), active);
    }

    private void readObject(ObjectInputStream in) throws IOException, ClassNotFoundException
        { in.defaultReadObject();
        lock = new Object();
    }
}
```

- **Output :-**

```
PS D:\CSPIT\Fenil\OOP\Mini_Bank> javac -d out (Get-ChildItem -Recurse src/main/j
>> java -cp out MiniBank
BankInfo[name=MiniBank, branch=CSPIT Branch]

1.Open 2.Deposit 3.Withdraw 4.Transfer 5.Working hours 6.Exit
Choice: 1
Owner name: Fenil
Opening balance: 2000
Opened SavingsAccount{number=AC0001, owner=Fenil, balance=2000, active=true}

1.Open 2.Deposit 3.Withdraw 4.Transfer 5.Working hours 6.Exit
Choice: 2
Account number: AC001
Account not found

1.Open 2.Deposit 3.Withdraw 4.Transfer 5.Working hours 6.Exit
Choice: 2
Account number: AC0001
Amount: 500
--- Account Statement ---
Number: AC0001
Owner: Fenil
Balance: Rs. 2500
Status: Active


1.Open 2.Deposit 3.Withdraw 4.Transfer 5.Working hours 6.Exit
Choice: 3
Account number: AC0001
Amount: 200
--- Account Statement ---
Number: AC0001
Owner: Fenil
Balance: Rs. 2300
Status: Active


1.Open 2.Deposit 3.Withdraw 4.Transfer 5.Working hours 6.Exit
Choice: 6
Goodbye from MiniBank.
```

Learning Outcome:

The learning outcome of this practical is to understand and apply Java classes, constructors, constructor chaining, static and final variables, methods, conditional statements, and loops. Students will learn to manage object state, control values within specified limits, perform booking and cancellation operations, maintain shared counters, and access data using getter methods.

PRACTICAL - 3

Aim: To develop a Java program to identify and count distinct points using classes, equals(), hashCode(), and toString() methods.

Code Screenshot:

- **distinct_point:**

  Driver.java

Point.java 5 Driver.java

distinct_point > Driver.java > Java > Driver > main(String[] args)

```
public class Driver {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {

        Point[] points = {
                new Point(x: 1, y: 2),
                new Point(x: 3, y: 4),
                new Point(x: 1, y: 2),
                new Point(x: 5, y: 6),
                new Point(x: 3, y: 4)
        };

        int distinct = 0;

        for (int i = 0; i < points.length; i++) {
            boolean found = false;
            for (int j = 0; j < i; j++) {
                if (points[i].equals(points[j])) {
                    found = true;
                    break;
                }
            }
            if (!found) {
                distinct++;
            }
        }
        System.out.println("Distinct: " + distinct);
    }
}
```

pointer.java

Point.java 5 Driver.java

distinct_point > Point.java > Java > Point

```java
import java.util.Objects;

public class Point {
    private int x;
    private int y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public String toString() {
        return "(" + x + ", " + y + ")";
    }

    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null || getClass() != obj.getClass()) {
            return false;
        }
        Point other = (Point) obj;
        return this.x == other.x && this.y == other.y;
    }

    public int hashCode() {
        return Objects.hash(x, y);
    }
}
```

- **duplicate_card_check:**

Card.java

Card.java 2 | Driver.java

Card.java > Language Support for Java(TM) by Red Hat > Card

```java
public class Card {

    private String rank;
    private String suit;

    public Card(String rank, String suit) {
        this.rank = rank;
        this.suit = suit;
    }

    @Override
    public String toString() {
        return rank + " of " + suit;
    }

    @Override
    public boolean equals(Object obj) {

        if (this == obj) {
            return true;
        }

        if (obj == null || getClass() != obj.getClass()) {
            return false;
        }

        Card card = (Card) obj;

        return rank.equals(card.rank) && suit.equals(card.suit);
    }

    @Override
    public int hashCode() {
        return rank.hashCode() + suit.hashCode();
    }
}
```

Driver.java

Card.java 2 Driver.java

Driver.java > Language Support for Java(TM) by Red Hat > Driver > main(String[])

```
1   public class Driver {
2
        Run | Debug | Run main | Debug main
3       public static void main(String[] args) {
4
5           Card[] cards = new Card[5];
6
7           cards[0] = new Card(rank: "Ace", suit: "Spades");
8
9           cards[1] = new Card(rank: "King", suit: "Hearts");
10
11          cards[2] = new Card(rank: "Queen", suit: "Diamonds");
12
13          cards[3] = new Card(rank: "Ace", suit: "Clubs");
14
15          cards[4] = new Card(rank: "Ace", suit: "Spades");
16
17          for (int i = 0; i < cards.length; i++) {
18
19              for (int j = 0; j < i; j++) {
20
21                  if (cards[i].equals(cards[j])) {
22                      System.out.println("Duplicate found: " + cards[i]);
23                      return;
24                  }
25              }
26          }
27      }
28  }
```

Output Screenshot:

- **distinct_points:**

```
PS D:\CSPIT\OOP\Practical_3> java .\distinct_point\Driver.java
Distinct: 3
PS D:\CSPIT\OOP\Practical_3>
```

- **duplicate_card_check:**

```
PS D:\CSPIT\OOP\Practical_3\duplicate_card_check> java .\Driver.java
Duplicate found: Ace of Spades
PS D:\CSPIT\OOP\Practical_3\duplicate_card_check>
```

PART - B

- **Customer.java**

```
package model;


import java.io.Serializable;
import model.annotation.MaxLength;


public class Customer implements Cloneable, Serializable {


    private static final long serialVersionUID = 1L;
    private static long customerCounter = 101;
    private final String customerId;
    @MaxLength(10)
    private String name;
    private final String email, mobile;
    private final Address address;


    public Customer(String name, String email, String mobile, Address address)
        { this.customerId = generateCustomerId();
        this.name = name;
        this.email = email;
        this.mobile = mobile;
        this.address = address == null ? null : new Address(address.line, address.city, address.pincode);
    }
```

```
    private static synchronized String generateCustomerId()
        { return "CUST" + customerCounter++;
    }

    public String getCustomerId()
        { return customerId;
    }

    public String getName()
        { return name;
    }

    public String getEmail()
        { return email;
    }

    public String getMobile()
        { return mobile;
    }

    public Address getAddress() {
        return address == null ? null : new Address(address.line, address.city, address.pincode);
    }

    @Override
```

```
    public Customer clone()
      { try {
          return (Customer) super.clone();
      } catch (CloneNotSupportedException e)
          { throw new AssertionError(e);
      }
    }

    public static class Address implements Serializable {

      private static final long serialVersionUID = 1L;
      private final String line, city, pincode;

      public Address(String line, String city, String pincode)
          { this.line = line;
          this.city = city;
          this.pincode = pincode;
      }

      public String getLine()
          { return line;
      }

      public String getCity()
          { return city;
```

```
    }

    public String getPincode()
      { return pincode;
    }
  }
}
```

- **Output:-**

```
PS D:\CSPIT\Fenil\OOP\Mini_Bank> javac -d out (Get-ChildItem -Recurse src/main/java,
PS D:\CSPIT\Fenil\OOP\Mini_Bank> java -cp out EvidenceChecks
Customer ID: CUST101
Clone has independent immutable address view: true
Account equality: same=true, different=false
Account text: SavingsAccount{number=AC0001, owner=Riya, balance=1000, active=true}
Deposit/withdraw balance: 1150
Expected checked exception: Withdrawal denied by account policy
Expected parser error: Expected TYPE ACCOUNT AMOUNT
Savings yearly interest: 46.0
```

Learning Outcome:

The learning outcome of this practical is to understand and apply Java classes, constructors, arrays, toString(), equals(), and hashCode() methods. Students will learn how to compare objects based on their data, identify repeated objects, count distinct elements, and detect duplicate cards using object equality.

PRACTICAL - 4

Aim: 1. Password strength (PasswordChecker.java + Driver.java)

(a) In PasswordChecker, check four rules — length ≥ 8, contains an uppercase letter, contains a digit, contains a special character — using regex (e.g. ".*[A-Z].*") or String methods.

(b) Write strength(String pw) that counts how many rules pass and returns "Weak" (0– 1), "Medium" (2–3) or "Strong" (4).

(c) In Driver.main: test several passwords; for each, print which rules passed and the final label.Expected output: "abc" → Weak; "Abcd1234!" → Strong.2

Code Screenshot:

PasswordChecker.java

```java
public class PasswordChecker {
    public static String strength(String pw) {
        int count = 0;

        if (pw.length() >= 8) {
            count++;
        }
        if (pw.matches(regex: ".*[A-Z].*")) {
            count++;
        }
        if (pw.matches(regex: ".*[0-9].*")) {
            count++;
        }
        if (pw.matches(regex: ".*[^a-zA-Z0-9].*")) {
            count++;
        }
        if (count <= 1) {
            return "Weak";
        } else if (count <= 3) {
            return "Medium";
        } else {
            return "Strong";
        }
    }
}
```

Driver.java

Output Screenshot:

```
 1   public class driver_4 {
PS C:\Users\BHAKTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> javac PasswordChecker.java
PS C:\Users\BHAKTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> javac driver_4.java
PS C:\Users\BHAKTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> java driver_4
bhakti -> Weak
bhakti24 -> Medium
Bhakti@3124 -> Strong
PS C:\Users\BHAKTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS>
```

```
 9
10          for (String pw : passwords) {
11              System.out.println(pw + " -> " + PasswordChecker.strength(pw));
12          }
13      }
14  }
```

Learning Outcome:

To understand and implement Java classes, methods, string handling, and conditional logic for validating password security rules. The program helps in checking password characteristics, calculating password strength, and classifying passwords as Weak, Medium, or Strong based on the number of rules satisfied.

Aim : 2. Chat-log filter (ChatFilter.java + Driver.java)

(a) In Driver.main, define a String[] of log lines like "10:05 alice Hello there" (include one malformed line with fewer than 3 parts).

(b) Read a keyword from the user.

(c) For each line, split into time/user/message with split(" ", 3); skip lines that do not have 3 parts.

(d) If the message contains the keyword (case-insensitive), increment a counter and append "time user: message" to a StringBuilder.

(e) Print "Matches: N" and the built report.Expected output: keyword "hello" over 3 lines (1 malformed) → "Matches: 1" and the matching line.3.

Code Screenshot:

ChatFilter.java

```java
public class ChatFilter {
    public static String filter(String[] logs, String keyword) {

        int count = 0;
        StringBuilder report = new StringBuilder();

        for (String line : logs) {
            String[] parts = line.split(regex: " ", limit: 3);

            if (parts.length < 3) {
                continue;
            }

            String time = parts[0];
            String user = parts[1];
            String message = parts[2];

            if (message.toLowerCase().contains(keyword.toLowerCase())) {
                count++;
                report.append(time)
                      .append(str: " ")
                      .append(user)
                      .append(str: ": ")
                      .append(message)
                      .append(str: "\n");
            }
        }
        return "Matches: " + count + "\n" + report;
    }
}
```

Driver.java

```java
import java.util.Scanner;

public class driver_4b {

    Run | Debug
    public static void main(String[] args) {

        String[] logs = {
            "10:05 bhakti Hello there",
            "10:06 dhruva How are you",
            "10:07"
        };

        Scanner input = new Scanner(System.in);

        System.out.print(s: "Enter keyword: ");
        String keyword = input.nextLine();

        System.out.println(ChatFilter.filter(logs, keyword));

        input.close();
    }
}
```

Output Screenshot:

```
PS C:\Users\BHAKTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> java driver_4b
Enter keyword: Hello
Matches: 1
10:05 bhakti: Hello there

PS C:\Users\BHAKTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> java driver_4b
Enter keyword:
Matches: 2
10:05 bhakti: Hello there
10:06 dhruva: How are you

PS C:\Users\BHAKTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> java driver_4b
Enter keyword: hiii
Matches: 0
```

PART - B

- **util/Validator.java**

```
package util;

import java.util.regex.Pattern;

public final class Validator {

    private Validator() {
    }
    private static final Pattern MOBILE = Pattern.compile("^[6-9][0-9]{9}$"), EMAIL =
Pattern.compile("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$"), PAN =
Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$"), IFSC = Pattern.compile("^[A-Z]{4}0[A-Z0-
9]{6}$"), AMOUNT = Pattern.compile("^[1-9][0-9]*$");

    public static boolean isValidMobile(String s) {
        return s != null && MOBILE.matcher(s).matches();
    }

    public static boolean isValidEmail(String s) {
        return s != null && EMAIL.matcher(s).matches();
    }

    public static boolean isValidPan(String s)
        {  return s != null &&
        PAN.matcher(s).matches();
    }

    public static boolean isValidIfsc(String s)
        {  return s != null &&
        IFSC.matcher(s).matches();
    }

    public static boolean isValidAmount(String s) {
        return s != null && AMOUNT.matcher(s).matches();
    }
}
```

- **Output:-**

```
Mobile valid/invalid: true/false
Parsed command: Command[type=DEPOSIT, accountNumber=AC0001, amount=500]
```

Learning Outcome:

To understand and implement Java string manipulation, arrays, Scanner, split(), StringBuilder, and conditional statements. The program helps in processing chat-log data, handling malformed input, performing case-insensitive keyword searches, and generating a filtered chat report efficiently.

PRACTICAL - 5

Aim: This practical demonstrates abstraction, inheritance, constructors, arrays, and polymorphism in Java. Abstract classes and methods are used to define common behavior, while subclasses provide specific implementations for shape areas and employee salaries.

Code Screenshot:

- **shape_areas:**
  Shape_Test.java



```
package Practical 5;

abstract class Shape {
    String name;
    Shape(String name) {
        this.name = name;
    }

    abstract double area();
}

class Circle extends Shape {
    private double radius;
    Circle(String name, double radius) {
        super(name);
        this.radius = radius;
    }

    double area() {
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape {
    private double length;
    private double width;
    Rectangle(String name, double length, double width) {
        super(name);
        this.length = length;
        this.width = width;
    }

    double area() {
        return length * width;
    }
}

class Triangle extends Shape {
    private double base;
    private double height;
    Triangle(String name, double base, double height) {
        super(name);
        this.base = base;
        this.height = height;
    }

    double area() {
        return 0.5 * base * height;
    }
}

public class Shape_Test {
    run | Debug | Run main | Debug main
    public static void main(String[] args) {

        Shape[] shapes = {
            new Circle(name: "Circle", radius: 5),
```


Practical_5 > Shape_Test.java > Language Support for Java(TM) by Red Hat > Shape > Shape(String)
public class Shape_Test {
Run | Debug | Run main | Debug main
public static void main(String[] args) {
Shape[] shapes = {
new Circle(name: "Circle", radius: 5),
new Rectangle(name: "Rectangle", length: 10, width: 4),
new Triangle(name: "Triangle", base: 8, height: 6),
new Circle(name: "Circle", radius: 3),
new Rectangle(name: "Rectangle", length: 7, width: 5)
};
double total = 0;
double largest = 0;
System.out.println(x: "Shape Areas:");
for (Shape shape : shapes) {
double currentArea = shape.area();
System.out.println("Area of " + shape.name + " = " + String.format(format: "%.2f", currentArea)
);
total += currentArea;
if (currentArea > largest) {
largest = currentArea;
}
}
System.out.println(x: "----------");
System.out.println("Total Area = " + String.format(format: "%.2f", total));
System.out.println("Largest Area = " + String.format(format: "%.2f", largest));
}
}

- **Payroll:**
  Payroll.java



```java
public class Payroll {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {

        Employee[] employees = {
                new FullTime(name: "Yash", id: 101, fixedSalary: 50000),
                new PartTime(name: "Rahul", id: 102, hours: 80, rate: 300),
                new Intern(name: "Amit", id: 103, stipend: 15000),
                new FullTime(name: "Jay", id: 104, fixedSalary: 45000),
                new Intern(name: "Karan", id: 105, stipend: 12000)
        };

        double total = 0;
        System.out.println(x: "Employee Payroll:");

        for (Employee employee : employees) {
            double salary = employee.monthlySalary();
            System.out.println("Name: " + employee.name + ", ID: " + employee.id +
                               ", Salary: " + String.format(format: "%.2f", salary));
            if (employee instanceof Intern) {
                System.out.println("Note => " + employee.name + " is an Intern.\n");
            }
            total += salary;
        }

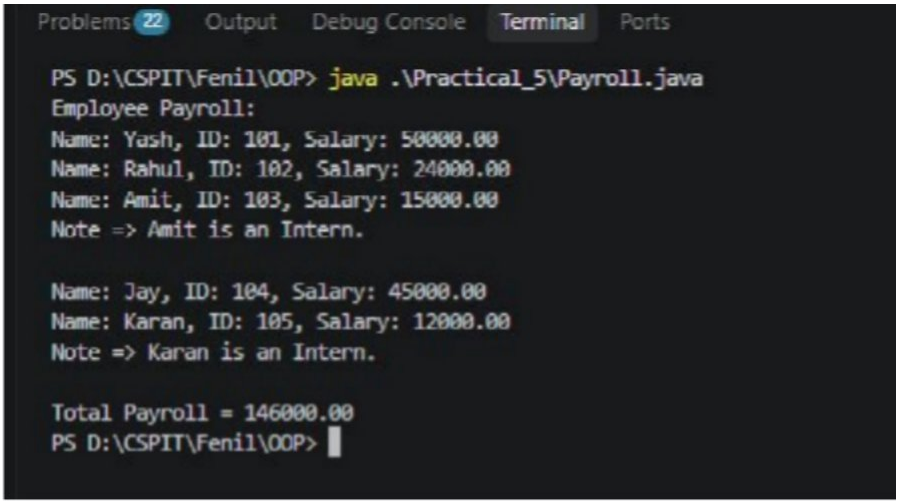
        System.out.println("Total Payroll = " + String.format(format: "%.2f", total));
    }
}
```

Output Screenshot:

- **Shape_Test:**



- **Payroll:**



PART – B

- **SavingsAccount.java**

```
package model;


public class SavingsAccount extends Account implements InterestBearing {


   private static final long serialVersionUID = 1L;
   private final long minBalance;


   public SavingsAccount(String owner, long opening, long minBalance)
      { super(owner, opening);
      if (minBalance < 0) {
         throw new IllegalArgumentException("Minimum balance cannot be negative");


      }this.minBalance = minBalance;
   }


   public long getMinBalance()
      { return minBalance;
   }


   @Override
   public double interestRate()
      { return 4.0;
```

```
    }

    @Override
    public boolean canWithdraw(long amount) {
        return amount > 0 && balanceUnsafe() - amount >= minBalance;
    }
}
```

- **Output:-**

```
SavingsAccount{number=AC0001, owner=Asha, balance=5000, active=true}, rate=4.0
CurrentAccount{number=AC0002, owner=Bharat, balance=2000, active=true}, rate=0.0
FixedDepositAccount{number=AC0003, owner=Chirag, balance=10000, active=true}, rate=7.0
Savings minimum balance: 1000
```

Learning Outcome:

The learning outcome of this practical is to understand and apply abstract classes, abstract methods, inheritance, constructors, arrays, and polymorphism in Java. Students will learn to process different objects through a common parent reference, calculate shape areas and employee salaries, find totals and maximum values, and use instance of to identify specific object types.

PRACTICAL - 6

Aim: 1. Remote control: an interface Switchable with on()/off() and a default toggle(); Fan and Light implement it. Loop over a Switchable[] toggling each. Then add a functional interface that decides, given a device and an hour, whether it may switch on, and supply it once as an anonymous class and once as a lambda.

Code Screenshot:

RemoteControl.java

```java
interface Switchable {
    void on();
    void off();
    default void toggle() {
        System.out.println("Toggle operation");
    }
}
class Fan implements Switchable {
    boolean isOn = false;
    @Override
    public void on() {
        isOn = true;
        System.out.println("Fan is ON.");
    }
    @Override
    public void off() {
        isOn = false;
        System.out.println("Fan is OFF.");
    }
    @Override
    public void toggle() {
        if (isOn) {
            off();
        } else {
            on();
        }
    }
}
class Light implements Switchable {
    boolean isOn = false;
    @Override
    public void on() {
        isOn = true;
        System.out.println("Light is ON.");
    }
    @Override
    public void off() {
        isOn = false;
        System.out.println("Light is OFF.");
    }
    @Override
    public void toggle() {
        if (isOn) {
            off();
        } else {
            on();
        }
    }
}
@FunctionalInterface
interface SwitchPermission {
    boolean maySwitchOn(Switchable device, int hour);
}
public class RemoteControl {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        // Array of Switchable devices
        Switchable[] devices = {
```

```java
RemoteControl.java > Java > RemoteControl
interface SwitchPermission {
}
public class RemoteControl {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        // Array of Switchable devices
        Switchable[] devices = {
            new Fan(),
            new Light()
        };
        // Toggle each device
        for (Switchable device : devices) {
            device.toggle();
        }
        // Anonymous class
        SwitchPermission permission1 = new SwitchPermission() {
            @Override
            public boolean maySwitchOn(Switchable device, int hour) {
                return hour >= 6 && hour <= 24;
            }
        };
        System.out.println(
            "Anonymous class: " +
            permission1.maySwitchOn(devices[1], hour: 18)
        );
        // Lambda expression
        SwitchPermission permission2 =
            (device, hour) -> hour >= 6 && hour <= 22;
        System.out.println(
            "Lambda: " +
            permission2.maySwitchOn(devices[0], hour: 6)
        );
    }
}
```

Output Screenshot:

```
PS D:\CSPIT\Fenil\OOP\Practical_6> javac .\RemoteControl.java
PS D:\CSPIT\Fenil\OOP\Practical_6> java RemoteControl
Fan is ON.
Light is ON.
Anonymous class: true
Lambda: true
PS D:\CSPIT\Fenil\OOP\Practical_6>
```

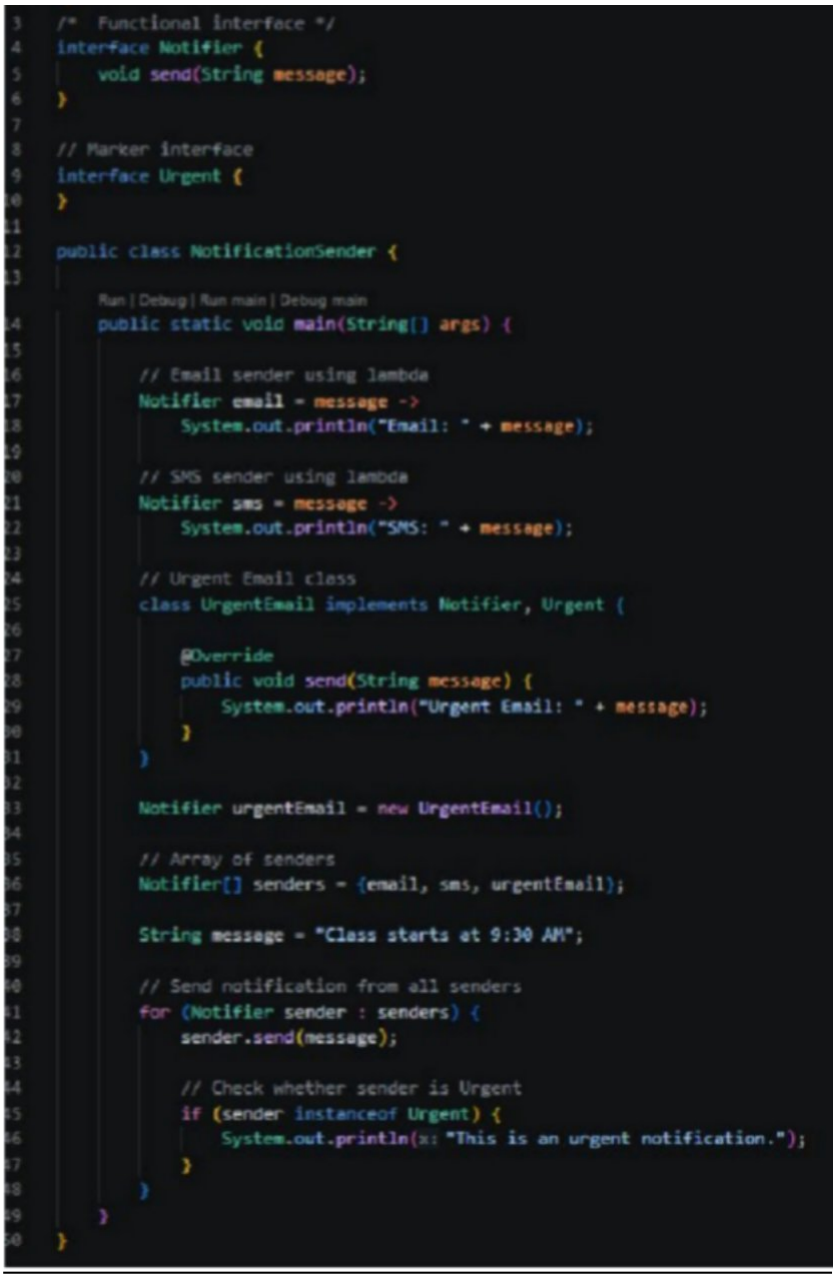
Learning Outcome:

To understand and implement interfaces, default methods, method overriding, polymorphism, functional interfaces, anonymous classes, and lambda expressions in Java. The program demonstrates how different devices such as Fan and Light implement a common Switchable interface, use the toggle() method through an array of Switchable objects, and define switching permissions using a functional interface implemented by both an anonymous class and a lambda expression.

Aim : 2. Notification senders: a functional interface Notifier with send(String); make email and SMS senders as lambdas, hold them in an array, and broadcast one message through all. Add a marker interface Urgent and send twice for senders marked urgent.

Code Screenshot:

NotificationSender.java



```java
/* Functional Interface */
interface Notifier {
    void send(String message);
}

// Marker interface
interface Urgent {
}

public class NotificationSender {

    Run | Debug | Run main | Debug main
    public static void main(String[] args) {

        // Email sender using lambda
        Notifier email = message ->
            System.out.println("Email: " + message);

        // SMS sender using lambda
        Notifier sms = message ->
            System.out.println("SMS: " + message);

        // Urgent Email class
        class UrgentEmail implements Notifier, Urgent {

            @Override
            public void send(String message) {
                System.out.println("Urgent Email: " + message);
            }
        }

        Notifier urgentEmail = new UrgentEmail();

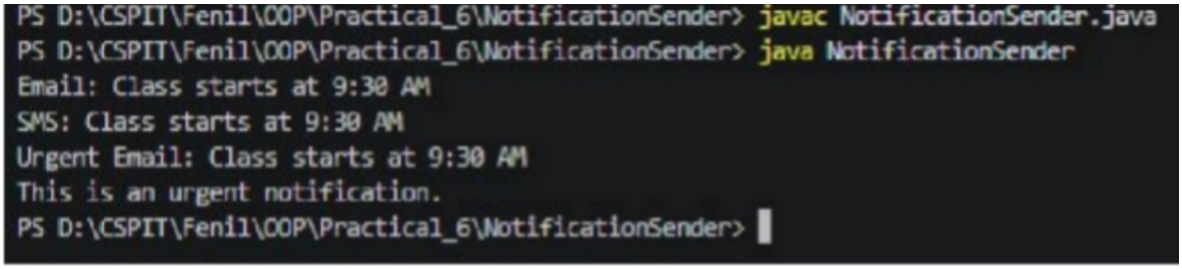
        // Array of senders
        Notifier[] senders = {email, sms, urgentEmail};

        String message = "Class starts at 9:30 AM";

        // Send notification from all senders
        for (Notifier sender : senders) {
            sender.send(message);

            // Check whether sender is Urgent
            if (sender instanceof Urgent) {
                System.out.println(x: "This is an urgent notification.");
            }
        }
    }
}
```

Output Screenshot:



PART - B

- **Model/InterestBearing.java**

```
package model;
public interface InterestBearing
    { double interestRate();
    long getBalance();
    default double yearlyInterest() { return getBalance() * interestRate() / 100.0; }
}
```

- **Output:-**



Learning Outcome:

To understand and implement functional interfaces, lambda expressions, arrays of interface objects, and marker interfaces in Java. The program demonstrates how Email and SMS notification senders can implement a common Notifier functional interface using lambda expressions, store them in an array, and broadcast a message to all senders. It also demonstrates the use of the Urgent marker interface to identify urgent senders and send the notification twice to them.

PRACTICAL - 7

Aim : 1. Form validator: define @NotBlank and @MaxLength(int) for fields and annotate a Signup Form. Write a checker that uses reflection to read each field's annotations and returns the list of validation errors for a given object.

Code Screenshot: FormValidator.java

```
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.List;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotBlank {
}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength {
    int value();
}
// Signup form class
class SignupForm {
    @NotBlank
    @MaxLength(20)
    String name;

    @NotBlank
    @MaxLength(30)
    String email;

    @NotBlank
    @MaxLength(10)
    String password;

    SignupForm(String name, String email, String password) {
        this.name = name;
        this.email = email;
        this.password = password;
    }
}

// Validator class
class FormValidator {

    public static List<String> validate(Object obj) {
        List<String> errors = new ArrayList<>();

        Field[] fields = obj.getClass().getDeclaredFields();

        for (Field field : fields) {
            field.setAccessible(true);

            try {
                Object value = field.get(obj);

                // Check NotBlank annotation
                if (field.isAnnotationPresent(NotBlank.class)) {
                    if (value == null || value.toString().trim().isEmpty()) {
                        errors.add(field.getName() + " must not be blank");
                    }
                }
```

```
            // Check MaxLength annotation
            if (field.isAnnotationPresent(MaxLength.class)) {
                MaxLength maxLength = field.getAnnotation(MaxLength.class);

                if (value != null && value.toString().length() > maxLength.value()) {
                    errors.add(field.getName() + " must not exceed "
                            + maxLength.value() + " characters");
                }
            }

        } catch (IllegalAccessException e) {
            errors.add("Unable to validate field: " + field.getName());
        }
    }

    return errors;
}

Run | Debug
public static void main(String[] args) {
    SignupForm form = new SignupForm(name: "", email: "bhakti@example.com",
            password: "verylongpassword123");

    List<String> errors = validate(form);

    if (errors.isEmpty()) {
        System.out.println("Form is valid");
    if (errors.isEmpty()) {
        System.out.println("Form is valid");
    } else {
        System.out.println("Validation Errors:")
        for (String error : errors) {
            System.out.println(error);
        }
    }
}
}
```

Output Screenshot:

```
Validation Errors:
name must not be blank
password must not exceed 10 characters
```

Aim: Mini test runner: define a @Run annotation and mark some no-arg methods of a class with it. Use reflection to find and invoke only the @Run methods and report how many ran — the idea behind how JUnit finds @Test methods.

Code Screenshot: MiniTestRunner.java

```
import java.lang.annotation.*;
import java.lang.reflect.Method;

// Custom Run annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run {
}

// Class containing test methods
class MyTests {

    @Run
    public void testAddition() {
        System.out.println("Test Addition executed");
    }

    @Run
    public void testSubtraction() {
        System.out.println("Test Subtraction executed");
    }

    public void normalMethod() {
        System.out.println("This method should not run");
    }

    @Run
    public void testMultiplication() {
        System.out.println("Test Multiplication executed");
    }
}

// Mini test runner
public class MiniTestRunner {

    Run | Debug
    public static void main(String[] args) throws Exception {

        MyTests testObject = new MyTests();
        int count = 0;

        Method[] methods = MyTests.class.getDeclaredMethods();

        for (Method method : methods) {
            if (method.isAnnotationPresent(Run.class)
                    && method.getParameterCount() == 0) {

                method.invoke(testObject);
                count++;
            }
        }

        System.out.println("Total tests executed: " + count);
    }
}
```

Output Screenshot

```
Test Addition executed
Test Subtraction executed
Test Multiplication executed
Total tests executed: 3
```

Learning Outcome:

To understand the creation and use of custom annotations in Java. Learned how to use Reflection API to identify and execute methods annotated with @Run at runtime. Also understood how to check method parameters and count the total number of test methods executed.

PRACTICAL - 8

Aim: 1. Guarded calculator: read two numbers and an operator; throw a custom DivideByZeroException and catch invalid-number input separately, printing a clear message each time; loop until a valid calculation succeeds and use finally to log each attempt.

Code Screenshot:

GuardedCalculator.java

```
import java.util.Scanner;

class DivideByZeroException extends Exception {
    public DivideByZeroException(String message) {
        super(message);
    }
}

public class GuardedCalculator {
    public static double calculate(double A, double B, char OP)
            throws DivideByZeroException {

        switch (OP) {
            case '+':
                return A + B;

            case '-':
                return A - B;

            case '*':
                return A * B;

            case '/':
                if (B == 0) {
                    throw new DivideByZeroException("Cannot divide by zero.")
                }
                return A / B;
        }
        return 0;
    }
```

```java
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        boolean succeed = false;

        while (!succeed) {

            try {
                System.out.print("Enter first number: ");
                double A = sc.nextDouble();

                System.out.print("Enter second number: ");
                double B = sc.nextDouble();

                System.out.print("Enter operator (+, -, *, /): ")
                char OP = sc.next().charAt(0);

                double result = calculate(A, B, OP);
                System.out.println("Result = " + result);
                succeed = true;
            }

            catch (java.util.InputMismatchException e) {
                System.out.println("Invalid number input. Please enter numbers only.")
                sc.nextLine();
            }
            catch (DivideByZeroException e) {
                System.out.println("Error: " + e.getMessage());
            }
            finally {
                System.out.println("Attempt completed.");
            }
        }
        sc.close();
    }
}
```

Output Screenshot:

```
dhruvasavaliya@DHRUVAs-MacBook-Pro:~/Desktop/Charusat/OOPJ% javac GuardedCalculator.java
dhruvasavaliya@DHRUVAs-MacBook-Pro:~/Desktop/Charusat/OOPJ% java GuardedCalculator
Enter first number: 10
Enter second number: 0
Enter operator (+, -, *, /): /
Error: Cannot divide by zero.
Attempt completed.
Enter first number: 10
Enter second number: 30
Enter operator (+, -, *, /): -
Result = -20.0
Attempt completed.
```

Learning Outcome:

To understand and implement exception handling in Java using custom exceptions, try-catch-finally blocks, and loops. The program demonstrates how to create and throw a custom DivideByZeroException, handle invalid number input separately using NumberFormatException, use the finally block to log every calculation attempt, and repeatedly accept input until a valid calculation is successfully completed.

Aim : 2. Stock issue: a Warehouse where issue(item, qty) throws a custom checked OutOfStockException carrying the shortfall, and InvalidQuantityException for qty ≤ 0. Process a list of requests, catching and reporting each failure without stopping the run.

Code Screenshot:

StockIssue.java

```
class OutOfStockException extends Exception {
    int shortfall;

    OutOfStockException(int shortfall) {
        this.shortfall = shortfall;
    }
}
class InvalidQuantityException extends Exception {
    InvalidQuantityException(String message) {
        super(message);
    }
}

class Warehouse {

    int stock = 10;
    void issue(String item, int qty)
            throws OutOfStockException, InvalidQuantityException {
        if (qty <= 0) {
            throw new InvalidQuantityException(
                    "Invalid quantity: " + qty);
        }
        if (qty > stock) {
            throw new OutOfStockException(qty - stock);
        }
        stock = stock - qty;
        System.out.println("Issued " + qty + " " + item);
        System.out.println("Remaining stock: " + stock);
    }
}
```

```
32  public class StockIssue {
33
34      public static void main(String[] args) {
35
36          Warehouse w = new Warehouse();
37
38          String[] items = {"Pen", "Pen", "Pen", "Pen"};
39          int[] quantities = {3, 8, 0, 5};
40
41          for (int i = 0; i < items.length; i++) {
42
43              try {
44                  w.issue(items[i], quantities[i]);
45
46              } catch (OutOfStockException e) {
47                  System.out.println(
48                      "Out of stock. Shortfall: " + e.shortfall)
49
50              } catch (InvalidQuantityException e) {
51                  System.out.println(e.getMessage());
52              }
53
54              System.out.println();
55          }
56
57          System.out.println("All requests processed.");
58      }
```

Output Screenshot:

```
dhruvasavaliya@DHRUVAs-MacBook-Pro:~/Desktop/Charusat/OOPJ% javac StockIssue.java
dhruvasavaliya@DHRUVAs-MacBook-Pro:~/Desktop/Charusat/OOPJ% java StockIssue
Issued 3 Pen
Remaining stock: 7
Out of stock. Shortfall: 1
Invalid quantity: 0
Issued 5 Pen
Remaining stock: 2
All requests processed.
```

PART - B

- **Model/Account.java**

```
package model;

import exception.*;
import java.io.*;
import java.util.Objects;
import model.annotation.Id;
import model.annotation.Positive;

public abstract class Account implements Transactable, Serializable

   { private static final long serialVersionUID = 1L;
   private static long counter = 1;
   @Id
   private final String accountNumber;
   private final String ownerName;
   @Positive
   private long balance;
   private boolean active = true;
   private transient Object lock = new Object();

   protected Account(String ownerName, long openingBalance) {
      if (ownerName == null || ownerName.isBlank() || openingBalance < 0) {
         throw new IllegalArgumentException("Owner and non-negative opening balance are required");

      }this.accountNumber = nextNumber();
      this.ownerName = ownerName;
      this.balance = openingBalance;
   }

   private static synchronized String nextNumber()
      { return String.format("AC%04d", counter++);
   }

   public final String getAccountNumber()
      { return accountNumber;
   }

   public final String getOwnerName()
      { return ownerName;
   }

   public final long getBalance()
      { synchronized (lock)
         { return balance;
      }
   }

   public final boolean isActive()
      { return active;
   }

   protected final Object accountLock()
      { return lock;
   }
```

```
    protected final long balanceUnsafe()
        { return balance;
    }

    @Override
    public final void deposit(long amount) throws InvalidAmountException
        { synchronized (lock)
            { validateAmount(amount);
            balance = Math.addExact(balance, amount);
        }
    }

    @Override
    public final boolean withdraw(long amount) throws InsufficientFundsException, InvalidAmountException
        { synchronized (lock)
            { validateAmount(amount);
            if (!canWithdraw(amount)) {
                throw new InsufficientFundsException("Withdrawal denied by account policy", Math.max(0,
amount - Math.max(0, balance)));

            }balance -= amount;
            return true;
        }
    }

    public final void transfer(Account to, long amount) throws BankException
        { if (to == null) {
            throw new AccountNotFoundException("Destination account is required");

        }if (to == this) {
            throw new BankException("Cannot transfer to the same account");

        }Account first = this.accountNumber.compareTo(to.accountNumber) < 0 ? this : to;
        Account second = first == this ? to : this;
        synchronized (first.lock)
            { synchronized (second.lock)
                { try {
                    validateAmount(amount);
                    if (!canWithdraw(amount)) {
                        throw new InsufficientFundsException("Transfer denied by account policy", Math.max(0,
amount - Math.max(0, balance)));

                    }this.balance -= amount;
                    try {
                        to.balance = Math.addExact(to.balance, amount);
                    } catch (ArithmeticException e)
                        { this.balance += amount;
                        throw new BankException("Transfer rolled back", e);
                    }
                } finally {
                    /* locks release automatically */ }
            }
        }
    }

    private static void validateAmount(long amount) throws InvalidAmountException
        { if (amount <= 0) {
```

```
            throw new InvalidAmountException("Amount must be positive");

        }}

    public abstract double interestRate();

    public abstract boolean canWithdraw(long amount);

    public final double monthlyInterest() {
        return getBalance() * interestRate() / 1200.0;
    }

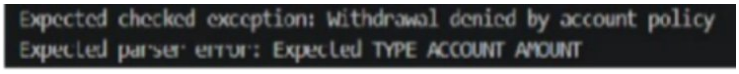
    @Override
    public final boolean equals(Object other) {
        return other instanceof Account a && accountNumber.equals(a.accountNumber);
    }

    @Override
    public final int hashCode() {
        return Objects.hash(accountNumber);
    }

    @Override
    public String toString() {
        return "%s{number=%s, owner=%s, balance=%d, active=%s}".formatted(getClass().getSimpleName(),
accountNumber, ownerName, getBalance(), active);
    }

    private void readObject(ObjectInputStream in) throws IOException, ClassNotFoundException
        { in.defaultReadObject();
        lock = new Object();
    }
}
```

- **Output:-**



Learning Outcome:

To understand and implement custom checked exceptions in Java for handling stock-related errors. The program demonstrates how to create and throw custom OutOfStockException carrying the required shortfall and InvalidQuantityException for invalid quantities, while processing multiple stock requests. It also demonstrates exception handling that reports each failure individually without stopping the execution of the remaining requests.

PRACTICAL - 9

Aim: 1. Counter race: several threads each increment a shared counter many times. Run it without synchronization to watch the total come out too low, then make the increment synchronized and confirm it is exactly right every run.

Code Screenshot:

CounterRace.java:-

Program 1A - Without Synchronization

```
class Counter {
    int count = 0;

    void increment() {
        count++;
    }
}
class CounterThread extends Thread {
    Counter counter;
    int times;

    CounterThread(Counter counter, int times) {
        this.counter = counter;
        this.times = times;
    }

    public void run() {
        for (int i = 0; i < times; i++) {
            counter.increment();
        }
    }
}

public class CounterRace {
    Run | Debug
    public static void main(String[] args) throws InterruptedException {

        Counter counter = new Counter();

        int numberOfThreads = 10;
        int incrementsPerThread = 100000;

        Thread[] threads = new Thread[numberOfThreads];

        for (int i = 0; i < numberOfThreads; i++) {
            threads[i] = new CounterThread(counter, incrementsPerThread);
            threads[i].start();
        }

        for (int i = 0; i < numberOfThreads; i++) {
            threads[i].join();
        }

        int expected = numberOfThreads * incrementsPerThread;

        System.out.println("Expected count: " + expected);
        System.out.println("Actual count:   " + counter.count);
    }
}
```

Output Screenshot:

```
PS C:\Users\BHAKTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> javac CounterRace.java
PS C:\Users\BHAKTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> java CounterRace
Expected count: 1000000
Actual count:   705384
PS C:\Users\BHAKTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS>
```

Program 1B - With Synchronization

```
    int count = 0;

    void increment() {
        count++;
    }
}

class CounterThread extends Thread {
    Counter counter;
    int times;

    CounterThread(Counter counter, int times) {
        this.counter = counter;
        this.times = times;
    }

    public void run() {
        for (int i = 0; i < times; i++) {
            counter.increment();
        }
    }
}

public class CounterRace_1 {
    Run | Debug
    public static void main(String[] args) throws InterruptedException {

        Counter counter = new Counter();

        int numberOfThreads = 10;
        int incrementsPerThread = 100000;

        Thread[] threads = new Thread[numberOfThreads];

        for (int i = 0; i < numberOfThreads; i++) {
            threads[i] = new CounterThread(counter, incrementsPerThread);
            threads[i].start();
        }

        // wait for all threads
        for (int i = 0; i < numberOfThreads; i++) {
            threads[i].join();
        }

        int expected = numberOfThreads * incrementsPerThread;

        System.out.println("Expected count: " + expected);
        System.out.println("Actual count:   " + counter.count);
    }
}
```

Output Screenshot:

```
PS C:\Users\BHAKTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> java CounterRace_1
Expected count: 1000000
Actual count:   424874
PS C:\Users\BHAKTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS>
```

Aim : 2 Seat booking race: a shared seatsLeft of 5 with about 10 threads each trying to book one seat. Without synchronization show that more than 5 can succeed (oversell); with a synchronized book() exactly 5 do.

2A. Without Synchronization

SeatBookingRace.java :-

```
class SeatBooking {
    int seatsLeft = 5;
    boolean book() {
        if (seatsLeft > 0) {
            try {
                Thread.sleep(millis: millis: 10);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            seatsLeft--;
            return true;
        }
        return false;
    }
}
class BookingThread extends Thread {
    SeatBooking booking;
    BookingThread(SeatBooking booking) {
        this.booking = booking;
    }
    public void run() {
        boolean success = booking.book();
        if (success) {
            System.out.println(
                Thread.currentThread().getName() +
                " booked a seat"
            );
        } else {
            System.out.println(
                Thread.currentThread().getName() +
                " failed - no seat"
            );
        }
    }
}
```

```
    public static void main(String[] args) throws InterruptedException {

        SeatBooking booking = new SeatBooking();

        int numberOfThreads = 10;

        Thread[] threads = new Thread[numberOfThreads];

        for (int i = 0; i < numberOfThreads; i++) {

            threads[i] = new BookingThread(booking);
            threads[i].setName("Customer " + (i + 1));

            threads[i].start();
        }

        for (int i = 0; i < numberOfThreads; i++) {
            threads[i].join();
        }

        System.out.println("Seats left: " + booking.seatsLeft);
    }
}
```

Output Screenshot:



2B. With Synchronization :

```
    int seatsLeft = 5;
    synchronized boolean book() {
        if (seatsLeft > 0) {
            try {
                Thread.sleep(millis: millis: 10);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            seatsLeft--;
            return true;
        }
        return false;
    }
}
class BookingThread extends Thread {
    SeatBooking booking;
    BookingThread(SeatBooking booking) {
        this.booking = booking;
    }
    public void run() {
        boolean success = booking.book();
        if (success) {
            System.out.println(
                Thread.currentThread().getName() +
                " booked a seat"
            );
        } else {
            System.out.println(
                Thread.currentThread().getName() +
                " failed - no seat"
            );
        }
    }
}

Run | Debug
public static void main(String[] args) throws InterruptedException

    SeatBooking booking = new SeatBooking();

    int numberOfThreads = 10;

    Thread[] threads = new Thread[numberOfThreads];

    for (int i = 0; i < numberOfThreads; i++) {
```

```
49              threads[i] = new BookingThread(booking);
50              threads[i].setName("Customer-" + (i + 1));
51
52              threads[i].start();
53          }
54
55          for (int i = 0; i < numberOfThreads; i++) {
56              threads[i].join();
57          }
58
59          System.out.println("Seats left: " + booking.seatsLeft)
60      }
61  }
62
```

Output Screenshot:

```
PS C:\Users\BHAKTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> javac SeatBookingRace_1.java
PS C:\Users\BHAKTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS> java SeatBookingRace_1
Customer-2 booked a seat
Customer-7 booked a seat
Customer-9 booked a seat
Customer-8 booked a seat
Customer-10 booked a seat
Customer-5 failed - no seat
Customer-6 failed - no seat
Customer-4 failed - no seat
Customer-1 failed - no seat
Customer-3 failed - no seat
Seats left: 0
PS C:\Users\BHAKTI PATEL\OneDrive\Desktop\CHARUSAT\OOPS>
```

PART – B

- **AccountWorker.java**

```
package service;

import exception.InvalidAmountException;
import model.Account;

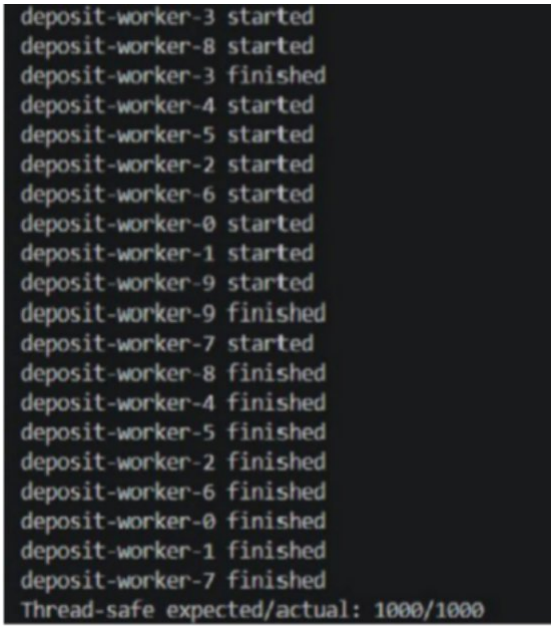
public class AccountWorker implements Runnable

    { private final Account account;
    private final int times;
    private final long amount;

    public AccountWorker(Account account, int times, long amount)
        { this.account = account;
        this.times = times;
        this.amount = amount;
    }

    @Override
    public void run()
        { System.out.println(Thread.currentThread().getName() + " started");
        for (int i = 0; i < times; i++)try {
            account.deposit(amount);
        } catch (InvalidAmountException e)
            { System.err.println("Worker error: " + e.getMessage());
            return;
        }
        System.out.println(Thread.currentThread().getName() + " finished");
    }
}
```

- **Output:-**


deposit-worker-3 started
deposit-worker-8 started
deposit-worker-3 finished
deposit-worker-4 started
deposit-worker-5 started
deposit-worker-2 started
deposit-worker-6 started
deposit-worker-0 started
deposit-worker-1 started
deposit-worker-9 started
deposit-worker-9 finished
deposit-worker-7 started
deposit-worker-8 finished
deposit-worker-4 finished
deposit-worker-5 finished
deposit-worker-2 finished
deposit-worker-6 finished
deposit-worker-0 finished
deposit-worker-1 finished
deposit-worker-7 finished
Thread-safe expected/actual: 1000/1000

Learning Outcome:

The three programs demonstrated the importance of thread synchronization when multiple threads access shared data. In the Counter Race program, unsynchronized access could produce an incorrect counter value, while using synchronized ensured the expected result every time. In the Seat Booking Race, lack of synchronization could cause more than the available 5 seats to be booked, whereas a synchronized book() method allowed exactly 5 successful bookings. The Advanced Array Sum program further showed how shared data can cause race conditions and how synchronization or separate partial results can solve the problem. Overall, these experiments demonstrated that synchronization prevents race conditions and maintains data consistency in multithreaded programs, although synchronization may introduce some performance overhead.

Practical – 10 (A1)

Aim: Thread-pool runner: with a fixed pool of 3 (ExecutorService), submit about 10 Runnable tasks that print their id and sleep briefly; shut down and await completion, observing the few threads being reused.

Source Code:

Thread_Pool_Runner.java

```
package Practical_10;
import java.util.concurrent.ExecutorService; import
java.util.concurrent.Executors;  import
java.util.concurrent.TimeUnit;

public class Thread_Pool_Runner { public static

    void main(String[] args) {
        // Create a fixed thread pool with 3 threads
        ExecutorService executor = Executors.newFixedThreadPool(3);

        // Submit 10 tasks
        for (int i = 1; i <= 10; i++) { final int
            taskId = i; executor.submit(() -> {
                System.out.println("Task " + taskId + " is running on "
+Thread.currentThread().getName());

                try {
                    // Sleep briefly to simulate some work
                    Thread.sleep(1000);
                } catch (InterruptedException e) { Thread.currentThread().interrupt();
                    System.out.println("Task " + taskId + " was interrupted.");
                }

                System.out.println("Task " + taskId + " completed by "
+Thread.currentThread().getName()
                );
            });
        }

        // No more tasks can be submitted
        executor.shutdown();

        try {
            // Wait until all tasks are completed
            if (executor.awaitTermination(30, TimeUnit.SECONDS))
                { System.out.println("\nAll tasks completed.");
            } else {
                System.out.println("\nTimeout occurred. Some tasks may still be running.");
            }
        } catch (InterruptedException e) { Thread.currentThread().interrupt();
            System.out.println("Main thread was interrupted.");
        }
    }
}
```

Output:

```
"C:\Program Files\Java\jdk-26.0.2\bin\java.exe" "-
Task 2 is running on pool-1-thread-2
Task 1 is running on pool-1-thread-1
Task 3 is running on pool-1-thread-3
Task 2 completed by pool-1-thread-2
Task 1 completed by pool-1-thread-1
Task 4 is running on pool-1-thread-1
Task 5 is running on pool-1-thread-2
Task 3 completed by pool-1-thread-3
Task 6 is running on pool-1-thread-3
Task 4 completed by pool-1-thread-1
Task 5 completed by pool-1-thread-2
Task 7 is running on pool-1-thread-1
Task 8 is running on pool-1-thread-2
Task 6 completed by pool-1-thread-3
Task 9 is running on pool-1-thread-3
Task 7 completed by pool-1-thread-1
Task 8 completed by pool-1-thread-2
Task 10 is running on pool-1-thread-1
Task 9 completed by pool-1-thread-3
Task 10 completed by pool-1-thread-1

All tasks completed.
```

Practical – 10 (A2)

Aim: Producer–consumer: one thread adds numbers to a small shared buffer and another removes them, coordinated with wait()/notify(); produce and consume a fixed number of items in order with none lost.

Source Code:

Producer_Consumer.java

```
package Practical_10; import
java.util.LinkedList; import
java.util.Queue;

public class Producer_Consumer {

    // Shared buffer
    static class SharedBuffer {
        private final Queue<Integer> buffer = new LinkedList<>(); private final int
        capacity = 3;

        // Add item to buffer
        public synchronized void produce(int item) throws InterruptedException {
            // Wait if buffer is full
            while (buffer.size() == capacity) { wait();
            }
            buffer.add(item);
            System.out.println("Produced: " + item + " | Buffer: " + buffer);
            // Notify consumer
            notifyAll();
        }

        // Remove item from buffer
        public synchronized int consume() throws InterruptedException {
            // Wait if buffer is empty
            while (buffer.isEmpty()) { wait();
            }
            int item = buffer.remove();
            System.out.println("Consumed: " + item + " | Buffer: " + buffer);
            // Notify producer
            notifyAll();
            return item;
        }
    }

    public static void main(String[] args) { SharedBuffer buffer
        = new SharedBuffer(); final int itemCount = 10;
        // Producer thread
        Thread producer = new Thread(() -> { try
                { for (int i = 1; i <= itemCount; i++)
                    { buffer.produce(i); Thread.sleep(200);
                }
            } catch (InterruptedException e) { Thread.currentThread().interrupt();
            }
        }, "Producer");

        // Consumer thread
        Thread consumer = new Thread(() -> { try
                { for (int i = 1; i <= itemCount; i++) {
                    buffer.consume();
                    Thread.sleep(400);
                }
```

```
            } catch (InterruptedException e) { Thread.currentThread().interrupt();
            }
        }, "Consumer");

        // Start both threads producer.start();
        consumer.start();

        // Wait for both threads to finish
        try {
            producer.join(); consumer.join();
        } catch (InterruptedException e)
            { Thread.currentThread().interrupt();
        }

        System.out.println("\nAll items produced and consumed successfully.");
    }
}
```

Output:

```
C:\Program Files\Java\jdk-25.0.2\bin\java.exe ...
Produced: 1 | Buffer: [1]
Consumed: 1 | Buffer: []
Produced: 2 | Buffer: [2]
Produced: 3 | Buffer: [2, 3]
Consumed: 2 | Buffer: [3]
Produced: 4 | Buffer: [3, 4]
Consumed: 3 | Buffer: [4]
Produced: 5 | Buffer: [4, 5]
Produced: 6 | Buffer: [4, 5, 6]
Consumed: 4 | Buffer: [5, 6]
Produced: 7 | Buffer: [5, 6, 7]
Consumed: 5 | Buffer: [6, 7]
Produced: 8 | Buffer: [6, 7, 8]
Consumed: 6 | Buffer: [7, 8]
Produced: 9 | Buffer: [7, 8, 9]
Consumed: 7 | Buffer: [8, 9]
Produced: 10 | Buffer: [8, 9, 10]
Consumed: 8 | Buffer: [9, 10]
Consumed: 9 | Buffer: [10]
Consumed: 10 | Buffer: []

All items produced and consumed successfully.
```

PART - B

- **TransactionBuffer.java**

```
package service;

import model.Command;

public final class TransactionBuffer {

   private final Command[] buffer;
   private int head, tail, count;
   private boolean closed;

    public TransactionBuffer(int capacity)
       { buffer = new Command[capacity];
   }

   public synchronized void put(Command c) throws InterruptedException
      { while (count == buffer.length && !closed)
         { wait();

      }if (closed) {
         throw new IllegalStateException("Buffer closed");

      }buffer[tail] = c;
      tail = (tail + 1) % buffer.length;
      count++;
      notifyAll();
   }

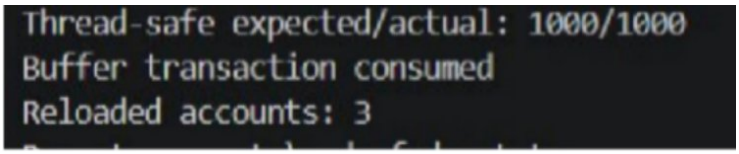
   public synchronized Command take() throws InterruptedException
      { while (count == 0 && !closed) {
         wait();
```

```
        }if (count == 0)
            { return null;


        }Command c = buffer[head];
        head = (head + 1) % buffer.length;
        count--;
        notifyAll();
        return c;
    }

    public synchronized void close()
        { closed = true;
        notifyAll();
    }
}
```

- **Output:-**



Learning Outcome:

- Understand multithreading and create multiple threads to perform tasks concurrently.
- Use ExecutorService and thread pools to manage and execute multiple tasks efficiently.
- Implement thread synchronization using synchronized, wait(), and notify().
- Develop a producer-consumer model using a shared buffer while ensuring no transactions are lost.
- Understand and identify deadlocks caused by inconsistent locking of shared resources.
- Prevent deadlocks by following a consistent lock-ordering strategy, such as locking the account with the smaller account number first.
- Apply multithreading concepts to MiniBank by processing deposits, withdrawals, and transfers safely and concurrently.
- Understand safe transaction processing in a banking system using concurrency and synchronization.

GitHub Repository:

https://github.com/Fenilrathod002/D26IT117-Fenil.git

PRACTICAL - 11

Aim: 1. Save/load: make a small class Serializable, save an array of its objects to a file with ObjectOutputStream, then read it back into fresh objects and confirm the values survived. Mark one field transient and show it is not saved.

Code:

SerializableRecord.java

```
import java.io.Serial;
import java.io.Serializable;

public class SerializableRecord implements Serializable
    { @Serial
    private static final long serialVersionUID = 1L;

    private final int id;
    private final String name;
    private final double score;
    private transient String sessionNote;

    public SerializableRecord(int id, String name, double score, String
sessionNote) {
        this.id = id;
        this.name = name;
        this.score = score;
        this.sessionNote = sessionNote;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public double getScore() { return score; }
    public String getSessionNote() { return sessionNote; }

    @Override
    public String toString() {
        return "Record{id=" + id + ", name='" + name + "', score=" + score
                + ", sessionNote=" + sessionNote + "}";
    }
}
```

SaveLoadDemo.java

```
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.nio.file.Files;
import java.nio.file.Path;

public class SaveLoadDemo {
    public static void main(String[] args) throws IOException,
ClassNotFoundException {
        Path file = Path.of("data", "records.dat");
        Files.createDirectories(file.getParent());

        SerializableRecord[] original = {
                new SerializableRecord(101, "Riya", 87.5, "active session"),
                new SerializableRecord(102, "Aarav", 92.0, "temporary note")
        };

        try (ObjectOutputStream output = new
ObjectOutputStream(Files.newOutputStream(file))) {
            output.writeObject(original);
        }

        SerializableRecord[] restored;
        try (ObjectInputStream input = new
ObjectInputStream(Files.newInputStream(file))) {
            restored = (SerializableRecord[]) input.readObject();
        }

        System.out.println("Saved records to: " + file.toAbsolutePath());
        System.out.println("Records after loading:");
        for (SerializableRecord record : restored)
            { System.out.println(record);
        }
        System.out.println("Transient sessionNote after loading: "
                + restored[0].getSessionNote());
    }
}
```

Output Screenshot:

```
Saved records to: /Users/dhruvasavaliya/Desktop/Charusat/OOP_Practicals/D26IT107-OOP/Practical-11/data/
records.dat
Records after loading:
Record{id=101, name='Riya', score=87.5, sessionNote=null}
Record{id=102, name='Aarav', score=92.0, sessionNote=null}
Transient sessionNote after loading: null
```

Learning Outcome:

I created a serializable class and saved an array of its objects to a file with ObjectOutputStream. I loaded the objects back with ObjectInputStream and confirmed their values were preserved. I marked the session note as transient and confirmed it was not restored.

Aim : 2. Log analyzer: create a few text files, then use Path/Files to read each line by line and count the total lines and how many contain a keyword across all files; also print each file's size from its attributes.

Code:

LogAnalyzer.java

```
import java.io.BufferedReader;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.List;
import java.util.Locale;

public class LogAnalyzer {
    private static final String KEYWORD = "ERROR";

    public static void main(String[] args) throws IOException
        { Path logsDirectory = Path.of("data", "logs");
        List<Path> files;
        try (var paths = Files.list(logsDirectory)) {
            files = paths.filter(Files::isRegularFile).sorted().toList();
        }

        int totalLines = 0;
        int matchingLines = 0;
        for (Path file : files) {
            BasicFileAttributes attributes = Files.readAttributes(file,
BasicFileAttributes.class);
            int fileLines = 0;
            int fileMatches = 0;
            try (BufferedReader reader = Files.newBufferedReader(file))
                { String line;
                while ((line = reader.readLine()) != null)
                    { fileLines++;
                    if
(line.toLowerCase(Locale.ROOT).contains(KEYWORD.toLowerCase(Locale.ROOT))) {
                        fileMatches++;
                    }
                }
            }
            totalLines += fileLines;
            matchingLines += fileMatches;
            System.out.printf("%s: %d bytes, %d lines, %d containing %s%n",
                    file.getFileName(), attributes.size(), fileLines,
fileMatches, KEYWORD);
        }

        System.out.println("Files analyzed: " + files.size());
        System.out.println("Total lines: " + totalLines);
```

```
        System.out.println("Lines containing " + KEYWORD + ": " +
matchingLines);
    }
}
```

server-a.txt

```
Practical-11 > data > logs >  server-a.txt
INFO Application started
ERROR Unable to connect to payment servic
INFO Retry scheduled
```

server-b.txt

```
Practical-11 > data > logs >  server-b.tx
INFO User signed in
WARN Disk usage is above 80 percent
ERROR Request timed out
```

Output Screenshot:

```
server-a.txt: 115 bytes, 3 lines, 1 containing ERROR
server-b.txt: 105 bytes, 4 lines, 1 containing ERROR
Files analyzed: 2
Total lines: 9
Lines containing ERROR: 2
```

Learning Outcome:

I created sample log files and used Path and Files to locate them. The program opens each file with a BufferedReader and reads it one line at a time. It counts the total lines and the lines containing the keyword ERROR, then uses BasicFileAttributes to display each file's size. This demonstrates basic file reading, keyword searching, resource management, and file attribute access with Java NIO.

MINIBANK

Aim: 1. Make Account (and its subclasses and Customer) implement Serializable, and add a private static final long serialVersionUID field.

2. Create a class named StatePersister with a public static method void save(Account[] accounts, Path file) that uses ObjectOutputStream inside try-with-resources, and a public static method Account[] load(Path file) that uses ObjectInputStream.

3. Create a class named TransactionLog with a public static method void append(Path file, String line) that uses Files.write(...) with CREATE and APPEND.

4. Create a class named ReportGenerator that walks the logs folder using Files.newDirectoryStream, reads each file line by line with a BufferedReader, and adds up the total deposits and total withdrawals in long variables; also read file details (size, last-modified) using Files.readAttributes, then write the summary to a report file.

5. In main, save the accounts, reload them into a new array and print them to confirm the data survived, then generate and print the report.

CODE (ReportGenerator.java)

```
package service;

import java.io.BufferedReader;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.DirectoryStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.Locale;

public final class ReportGenerator
   { private ReportGenerator() { }

   public static String generate(Path logsDirectory, Path reportFile) throws IOException
      { Files.createDirectories(logsDirectory);
      long totalDeposits = 0;
      long totalWithdrawals = 0;
      int malformedLines = 0;
      StringBuilder report = new StringBuilder("MiniBank End-of-Day Report\n");
      report.append("Log files:\n");

      try (DirectoryStream<Path> files = Files.newDirectoryStream(logsDirectory))
         { for (Path file : files) {
            if (!Files.isRegularFile(file))
               { continue;
            }
            BasicFileAttributes attributes =
                  Files.readAttributes( file,
                  BasicFileAttributes.class);
            report.append("  ").append(file.getFileName())
                  .append(" | ").append(attributes.size()).append(" bytes\n");

            try (BufferedReader reader = Files.newBufferedReader(file, StandardCharsets.UTF_8))
               { String line;
               while ((line = reader.readLine()) != null)
                  { String[] fields =
                  line.trim().split("\\s+"); if (fields.length
                  < 3) {
                     malformedLines++;
                     continue;
                  }
                  long amount;
                  try {
                     amount = Long.parseLong(fields[2]);
                  } catch (NumberFormatException e)
                     { malformedLines++;
                     continue;
                  }
                  if (amount < 0)
                     { malformedLines++;
                     continue;
                  }
                  switch (fields[0].toUpperCase(Locale.ROOT))
                     { case "DEPOSIT" -> totalDeposits += amount;
                     case "WITHDRAW" -> totalWithdrawals += amount;
                     default -> malformedLines++;
                  }
               }
            }
         }
```

```
    }
```

```
        report.append("Total deposits: ").append(totalDeposits).append(" rupees\n")
              .append("Total withdrawals: ").append(totalWithdrawals).append(" rupees\n")
              .append("Net change: ").append(totalDeposits - totalWithdrawals)
              .append(" rupees\n")
              .append("Unrecognized log lines: ").append(malformedLines).append('\n');

        Path parent = reportFile.toAbsolutePath().getParent();
        if (parent != null) {
            Files.createDirectories(parent);
        }
        Files.writeString(reportFile, report.toString(), StandardCharsets.UTF_8,
              StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING);
        return report.toString();
    }
}
```

CODE (StatePersister.java)

```
package service;

import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.nio.file.Files;
import java.nio.file.Path;
import model.Account;

public final class StatePersister
    { private StatePersister() { }

    public static void save(Account[] accounts, Path file) throws IOException
        { Path parent = file.toAbsolutePath().getParent();
        if (parent != null)
            { Files.createDirectories(parent);
        }
        try (ObjectOutputStream output = new ObjectOutputStream(Files.newOutputStream(file)))
            { output.writeObject(accounts);
        }
    }

    public static Account[] load(Path file) throws IOException, ClassNotFoundException
        { if (Files.notExists(file))
            { return new Account[0];
        }
        try (ObjectInputStream input = new ObjectInputStream(Files.newInputStream(file)))
            { Object savedState = input.readObject();
            if (!(savedState instanceof Account[] accounts)) {
                throw new IOException("Saved bank data has an unexpected format.");
            }
            return accounts;
        }
    }
}
```

CODE (ReportGenerator.java)

```
package service;

import java.io.BufferedReader;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.DirectoryStream;
import java.nio.file.Files;
```

```
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.Locale;

public final class ReportGenerator
   { private ReportGenerator() { }

   public static String generate(Path logsDirectory, Path reportFile) throws IOException
      { Files.createDirectories(logsDirectory);
      long totalDeposits = 0;
      long totalWithdrawals = 0;
      int malformedLines = 0;
      StringBuilder report = new StringBuilder("MiniBank End-of-Day Report\n");
      report.append("Log files:\n");

      try (DirectoryStream<Path> files = Files.newDirectoryStream(logsDirectory))
         { for (Path file : files) {
            if (!Files.isRegularFile(file))
               { continue;
            }
            BasicFileAttributes attributes =
                  Files.readAttributes( file,
                  BasicFileAttributes.class);
            report.append("  ").append(file.getFileName())
                  .append(" | ").append(attributes.size()).append(" bytes\n");

            try (BufferedReader reader = Files.newBufferedReader(file, StandardCharsets.UTF_8))
               { String line;
               while ((line = reader.readLine()) != null)
                  { String[] fields =
                  line.trim().split("\\s+"); if (fields.length
                  < 3) {
                     malformedLines++;
                     continue;
                  }
                  long amount;
                  try {
                     amount = Long.parseLong(fields[2]);
                  } catch (NumberFormatException e)
                     { malformedLines++;
                     continue;
                  }
                  if (amount < 0)
                     { malformedLines++;
                     continue;
                  }
                  switch (fields[0].toUpperCase(Locale.ROOT))
                     { case "DEPOSIT" -> totalDeposits += amount;
                     case "WITHDRAW" -> totalWithdrawals += amount;
                     default -> malformedLines++;
                  }
               }
            }
         }
      }

      report.append("Total deposits: ").append(totalDeposits).append(" rupees\n")
            .append("Total withdrawals: ").append(totalWithdrawals).append(" rupees\n")
            .append("Net change: ").append(totalDeposits - totalWithdrawals)
            .append(" rupees\n")
            .append("Unrecognized log lines: ").append(malformedLines).append('\n');
```

```
    Path parent = reportFile.toAbsolutePath().getParent();
    if (parent != null) {
```

```
            Files.createDirectories(parent);
        }
        Files.writeString(reportFile, report.toString(), StandardCharsets.UTF_8,
                StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING);
        return report.toString();
    }
}
```

OUTPUT

```
==========================================
MiniBank - Charusat Branch
==========================================

Main Menu
---------
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. List accounts
6. View statement
7. Run concurrency demo
8. Process transactions
9. Generate end-of-day report
10. Exit
Choose an option: 9
MiniBank End-of-Day Report
Log files:
  transactions.log | 61 bytes
Total deposits: 6400 rupees
Total withdrawals: 3400 rupees
Net change: 3000 rupees
Unrecognized log lines: 0
Report saved to data/reports/end-of-day-report.txt.
Main Menu
---------
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. List accounts
6. View statement
7. Run concurrency demo
8. Process transactions
9. Generate end-of-day report
10. Exit
Choose an option: 10
Thank you for banking with MiniBank. Goodbye!
Bank session closed.
Saved 3 account(s) to data/accounts.dat.
```

PRACTICAL - 12

Aim: 1. Word frequency: split a paragraph into words, count each in a HashMap, and print the words sorted by frequency (highest first) using a Comparator.

Code:

WordFrequency.java

```
import java.util.Comparator;
import java.util.HashMap;
import java.util.Locale;
import java.util.Map;

public class WordFrequency {
    public static void main(String[] args) {
        String paragraph = "Java makes reliable software. Java collections make
software "
                + "easier to organize, and reliable code is easier to
maintain.";
        Map<String, Integer> frequencies = new HashMap<>();

        for (String word : paragraph.toLowerCase(Locale.ROOT).split("[^\\p{L}\
\p{N}]+")) {
            if (!word.isBlank())
                { frequencies.merge(word, 1,
                Integer::sum);
            }
        }

        System.out.println("Word frequencies (highest first):");
        frequencies.entrySet().stream()
                .sorted(Map.Entry.<String,
Integer>comparingByValue(Comparator.reverseOrder())
                        .thenComparing(Map.Entry.comparingByKey()))
                .forEach(entry -> System.out.printf("%s: %d%n", entry.getKey(),
entry.getValue()));
    }
}
```

```
Word frequencies (highest first):
easier: 2
java: 2
reliable: 2
software: 2
to: 2
and: 1
code: 1
collections: 1
is: 1
maintain: 1
make: 1
makes: 1
organize: 1
```

Learning Outcome:

I used a paragraph as input and split it into individual words, ignoring punctuation and differences in letter case. I stored each word and its count in a `HashMap`, updating the count whenever the word appeared again. I then sorted the map entries with a `Comparator` so the most frequent words appear first. When words have equal counts, they are displayed alphabetically.

Aim : 2. Unique visitors: from a list of login names with repeats, use a HashSet to count unique users and a LinkedHashSet to keep first-seen order; print the unique count and the first three distinct names.

Code:

UniqueVisitors.java

```
import java.util.Arrays;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;

public class UniqueVisitors {
    public static void main(String[] args)
        { List<String> logins = Arrays.asList(
                "riya", "aarav", "diya", "riya", "kabir", "aarav", "meera");
        Set<String> uniqueUsers = new HashSet<>(logins);
        Set<String> firstSeenOrder = new LinkedHashSet<>(logins);

        System.out.println("Unique visitor count: " + uniqueUsers.size());
        System.out.println("First three distinct visitors:");
        firstSeenOrder.stream().limit(3).forEach(System.out::println);
    }
}
```

```
Unique visitor count: 5
First three distinct visitors:
riya
aarav
diya
```

Learning Outcome:

I used a list of login names that contains repeated visitors. A `HashSet` removes duplicates so the program can calculate the number of unique visitors. A `LinkedHashSet` also removes duplicates, while preserving the order in which each visitor first appeared. The program prints the unique count and the first three names in that order.

MINIBANK

Aim: 1. Create a class named AccountStore that keeps accounts in a HashMap<String, Account> using the account number as the key. Add methods add(Account account), get(String accountNumber) and getAll().

2. Keep a transaction history in an ArrayList<String>.

3. Add a method sortedByBalance() that returns an ArrayList<Account> sorted with a Comparator.

4. Make Account implement Comparable<Account> by comparing account numbers in compareTo().

5. In the TransactionProcessor, use a ConcurrentHashMap for the shared accounts so it is thread-safe.

6. In main, add several accounts to the store and print them sorted by balance.

CODE (AccountStore.java)

```
package service;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import model.Account;

public final class AccountStore {
    private final HashMap<String, Account> accounts = new HashMap<>();
    private final ArrayList<String> transactionHistory = new ArrayList<>();

    public void add(Account account)
        { if (account == null) {
            throw new IllegalArgumentException("Account cannot be null.");
        }
        accounts.put(account.getAccountNumber(),  account);
    }

     public Account get(String accountNumber)
         { return accounts.get(accountNumber);
    }

    public ArrayList<Account> getAll() {
        return new ArrayList<>(accounts.values());
    }

    /** Returns accounts from highest balance to lowest; account number breaks ties. */
    public ArrayList<Account> sortedByBalance() {
        ArrayList<Account> sortedAccounts = getAll();
        sortedAccounts.sort(Comparator.comparingDouble(Account::getBalance)
                .reversed().thenComparing(Account::getAccountNumber));
        return sortedAccounts;
    }

    public void recordTransaction(String transaction)
        { transactionHistory.add(transaction);
    }

     public ArrayList<String> getTransactionHistory()
         { return new ArrayList<>(transactionHistory);
    }
}
```

CODE (Account.java)

```
package model;

import java.io.Serial;
import java.io.Serializable;
import exception.AccountNotFoundException;
import exception.BankException;
import exception.InsufficientFundsException;
import exception.InvalidAmountException;
import model.annotation.Id;
import model.annotation.MaxLength;
import model.annotation.Positive;

public abstract class Account implements Transactable, InterestBearing,
```

```
    Serializable, Comparable<Account>

{ @Serial
private static final long serialVersionUID = 1L;

private static int nextAccountNumber = 1;

@Id
private final String accountNumber;
@MaxLength(50)
private String ownerName;
@Positive
protected double balance;

public Account(String ownerName) {
    this.accountNumber = String.format("AC%04d", nextAccountNumber++);
    this.ownerName = ownerName;
    this.balance = 0;
}

public Account(String ownerName, double balance) {
    this.accountNumber = String.format("AC%04d", nextAccountNumber++);
    this.ownerName = ownerName;
    this.balance = balance;
}

public String getAccountNumber()
    { return accountNumber;
}

public static void synchronizeAccountNumberSequence(Iterable<? extends Account> accounts)
    { int largestNumber = 0;
    for (Account account : accounts) {
        String number = account.getAccountNumber();
        if (number != null && number.startsWith("AC")) {
            try {
                largestNumber = Math.max(largestNumber,
                        Integer.parseInt(number.substring(2)));
            } catch (NumberFormatException ignored) {
                // Ignore account numbers that do not use the generated AC-number format.
            }
        }
    }
    nextAccountNumber = Math.max(nextAccountNumber, largestNumber + 1);
}

public String getOwnerName()
    { return ownerName;
}

public synchronized double getBalance()
    { return balance;
}

@Override
public synchronized void deposit(long amount) throws InvalidAmountException
    { if (amount <= 0) {
        throw new InvalidAmountException("Deposit amount must be greater than zero.");
    }
    balance += amount;
}
```

```
@Override
public synchronized boolean withdraw(long amount)
        throws InsufficientFundsException, InvalidAmountException
    { if (amount <= 0) {
        throw new InvalidAmountException("Withdrawal amount must be greater than zero.");
    }
    if (!canWithdraw(amount)) {
        long shortfall = Math.max(1, amount - Math.max(0, (long) balance));
        throw new InsufficientFundsException(shortfall);
    }
    balance -= amount;
    return true;
}

void depositUnsynchronizedForRaceDemo(long amount) throws InvalidAmountException
    { if (amount <= 0) {
        throw new InvalidAmountException("Deposit amount must be greater than zero.");
    }
    double currentBalance = balance;
    Thread.yield();
    balance = currentBalance + amount;
}

public void transfer(Account to, long amount) throws BankException
    { boolean withdrawn = false;
    boolean completed = false;
    try {
        if (to == null) {
            throw new AccountNotFoundException("destination");
        }
        if (equals(to)) {
            throw new InvalidAmountException("Source and destination must be different accounts.");
        }
        withdraw(amount);
        withdrawn = true;
        to.deposit(amount);
        completed = true;
    } catch (BankException e)
        { throw e;
    } finally {
        if (withdrawn && !completed)
            { balance += amount;
        }
    }
}

public abstract double interestRate();

public abstract boolean canWithdraw(long amount);

@Override
public synchronized String toString() {
    return accountNumber + " | " + ownerName + " | " + balance;
}

@Override
public boolean equals(Object o)
    { if (this == o)
        { return true;
    }

    if (o instanceof Account) {
```

```
            Account other = (Account) o;
            return this.accountNumber.equals(other.accountNumber);
        }

        return false;
    }

    @Override
    public int hashCode() {
        return accountNumber.hashCode();
    }

    @Override
    public int compareTo(Account other) {
        return accountNumber.compareTo(other.accountNumber);
    }
}
```

CODE (TransactionProcessor.java)

```
package service;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.ConcurrentHashMap;
import java.util.Collection;
import model.Account;

public final class TransactionProcessor {
    private final ExecutorService executor = Executors.newFixedThreadPool(4);
    private final ConcurrentHashMap<String, Account> sharedAccounts = new ConcurrentHashMap<>();

    public TransactionProcessor() { }

    public TransactionProcessor(Collection<? extends Account> accounts)
        { for (Account account : accounts)
            { registerAccount(account);
        }
    }

    public void registerAccount(Account account)
        { if (account == null) {
            throw new IllegalArgumentException("Account cannot be null.");
        }
        sharedAccounts.put(account.getAccountNumber(),  account);
    }

    public Account getAccount(String accountNumber)
        { return sharedAccounts.get(accountNumber);
    }

    public int getSharedAccountCount()
        { return sharedAccounts.size();
    }

    public void submit(Runnable task)
        { executor.execute(task);
    }
```

```
    public void stop() throws InterruptedException
        { executor.shutdown();
        if (!executor.awaitTermination(30, TimeUnit.SECONDS))
            { executor.shutdownNow();
            if (!executor.awaitTermination(30, TimeUnit.SECONDS)) {
                throw new IllegalStateException("Transaction workers did not stop.");
            }
        }
    }

    public void stopNow()
        { executor.shutdownNow();
    }
}
```

OUTPUT

```
./build.sh
java -jar build/minibank.jar
Built /Users/dhruvasavaliya/Desktop/Charusat/OOP_Practicals/D26IT107-OOP/MiniBank/build/minibank.jar
Loaded 3 saved account(s).
======================================
Account Validation
======================================
Invalid account: AC0004 | Validation Demo | -100.0
Validation errors:
  - balance must be > 0
Valid account: AC0001 | Dhruva | 8400.0
Validation errors: none

Accounts sorted by balance:
AC0003 | Aarya | 10000.0 | interest 7.0% per year | estimated yearly interest: 700.00
  Fixed deposit is locked; withdrawals are not allowed.
AC0001 | Dhruva | 8400.0 | interest 4.0% per year | estimated yearly interest: 336.00
AC0002 | Bhakti | 1600.0 | interest 0.0% per year | estimated yearly interest: 0.00
  Premium account

==============================
MiniBank - Charusat Branch
==============================

Main Menu
---------
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. List accounts
6. View statement
7. Run concurrency demo
8. Process transactions
9. Generate end-of-day report
10. View transaction history
11. Exit
```

```
Choose an option: 5

Account list:
AC0003 | Aarya | 10000.0 | interest 7.0% per year | estimated yearly interest: 700.00
  Fixed deposit is locked; withdrawals are not allowed.
AC0001 | Dhruva | 8400.0 | interest 4.0% per year | estimated yearly interest: 336.00
AC0002 | Bhakti | 1600.0 | interest 0.0% per year | estimated yearly interest: 0.00
  Premium account
Main Menu
---------
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. List accounts
6. View statement
7. Run concurrency demo
8. Process transactions
9. Generate end-of-day report
10. View transaction history
11. Exit
Choose an option: 11
Thank you for banking with MiniBank. Goodbye!
Bank session closed.
Saved 3 account(s) to data/accounts.dat.
```

PRACTICAL - 13

Aim: 1. Generic containers: a generic Box<T> with get/set and a Pair<A,B>; use them with different types to show type safety without casts, and write a generic method that returns the first element of any array.

Code:

Box.java

```
public class Box<T>
    { private T value;

    public Box(T value) { this.value = value; }
    public T get() { return value; }
    public void set(T value) { this.value = value; }
}
```

GenericContainersDemo.java

```
public class GenericContainersDemo
    { public static <T> T first(T[] values)
    {
        if (values == null || values.length == 0) {
            throw new IllegalArgumentException("Array must contain at least one
value.");
         }
        return values[0];
    }

    public static void main(String[] args) {
        Box<String> message = new Box<>("Generic types in Java");
        Box<Integer> count = new Box<>(3);
        Pair<String, Integer> accountSummary = new Pair<>("AC0001", 5000);

        System.out.println("String box: " + message.get());
        System.out.println("Integer box: " + count.get());
        System.out.println("Pair: " + accountSummary.first() + " -> " +
accountSummary.second());
        System.out.println("First string: " + first(new String[]{"Java",
"Generics"}));
        System.out.println("First integer: " + first(new Integer[]{10, 20,
30}));
    }
}
```

Pair.java

```
public record Pair<A, B>(A first, B second) { }
```

```
String box: Generic types in Java
Integer box: 3
Pair: AC0001 -> 5000
First string: Java
First integer: 10
```

Learning Outcome:

I created Box<T> to store a value of any chosen type and Pair<A, B> to store two values that may have different types. The demo uses these classes with values such as text and integers, showing how the compiler checks that values are used with their declared types. I also wrote a generic first method that returns the first element from an array. The same method works with arrays of different element types without requiring casts.

Aim : 2. Bounded stats: a generic method <N extends Number> double average(N[] values) that works for Integer[], Double[] and Long[]; note why it refuses non-Number types.

Code:

BoundedStats.java

```
public class BoundedStats {
    public static <N extends Number> double average(N[] values)
        { if (values == null || values.length == 0) {
            throw new IllegalArgumentException("Values must not be empty.");
        }
        double sum = 0.0;
        for (N value : values)
            { if (value == null)
            {
                throw new IllegalArgumentException("Values must not contain
null.");
            }
            sum += value.doubleValue();
        }
        return sum / values.length;
    }

    public static void main(String[] args) {
        System.out.println("Integer average: " + average(new Integer[]{10, 20,
30}));
        System.out.println("Double average: " + average(new Double[]{2.5, 3.5,
4.0}));
        System.out.println("Long average: " + average(new Long[]{100L, 200L,
300L}));
        System.out.println("The <N extends Number> bound rejects non-number
types at compile time.");
    }
```

```
}
```

```
Integer average: 20.0
Double average: 3.3333333333333335
Long average: 200.0
The <N extends Number> bound rejects non-number types at compile time.
```

Learning Outcome:

I wrote an average method with the type parameter <N extends Number>. The method accepts arrays whose elements are numeric types and converts each value to double to calculate the average. I demonstrated it with Integer[], Double[], and Long[]. The Number bound provides compile-time type safety: an array of non-numeric values cannot be passed to this method.

MINIBANK

Aim: 1. Create a generic class named Repository<T> that holds a HashMap<String, T>. Add methods T save(T entity), T findById(String id), ArrayList<T> findAll() and boolean delete(String id).

2. Inside save(), find the id by reflection: look for the field marked @Id (from Practical 7) and read its value as the map key.

3. Add a method marked @SafeVarargs called saveAll(T... items) that saves several objects at once.

4. Add one generic method with a bounded type parameter, for example static <N extends Number> double average(N[] values).

5. In main, create a Repository<Account>, save several accounts, fetch one by id, and print all of them.

CODE (Repository.java)

```
package service;
```

```
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.HashMap;
import model.annotation.Id;

public final class Repository<T> {
    private final HashMap<String, T> entities = new HashMap<>();

    public T save(T entity)
        { if (entity == null) {
            throw new IllegalArgumentException("Entity cannot be null.");
        }
        String id = readId(entity);
        entities.put(id, entity);
        return entity;
    }

    @SafeVarargs
    public final void saveAll(T... items)
        { if (items == null) {
            throw new IllegalArgumentException("Items cannot be null.");
        }
        for (T item : items)
            { save(item);
        }
    }

    public T findById(String id)
        { return entities.get(id);
    }

    public ArrayList<T> findAll() {
        return new ArrayList<>(entities.values());
    }

    public boolean delete(String id)
        { return entities.remove(id) != null;
    }

    public static <N extends Number> double average(N[] values)
        { if (values == null || values.length == 0) {
            throw new IllegalArgumentException("Values must contain at least one number.");
        }
        double total = 0;
        for (N value : values)
            { if (value == null) {
                throw new IllegalArgumentException("Values cannot contain null.");
            }
            total += value.doubleValue();
        }
        return total / values.length;
    }

    private String readId(T entity)
        { Field idField = null;
        Class<?> type = entity.getClass();
        while (type != null && type != Object.class)
            { for (Field field : type.getDeclaredFields())
                { if (field.isAnnotationPresent(Id.class))
                    { if (idField != null) {
                        throw new IllegalArgumentException("Entity has multiple @Id fields: "
```

```
                        + entity.getClass().getName());
                }
                idField = field;
            }
        }
        type = type.getSuperclass();
    }
    if (idField == null) {
        throw new IllegalArgumentException("Entity has no @Id field: "
                + entity.getClass().getName());
    }

    try {
        idField.setAccessible(true);
        Object value = idField.get(entity);
        if (value == null) {
            throw new IllegalArgumentException("Entity @Id value cannot be null.");
        }
        return value.toString();
    } catch (IllegalAccessException e) {
        throw new IllegalStateException("Cannot read the entity @Id field.", e);
    }
  }
}
```

CODE (Customer.java)

```
package model;

import java.io.Serial;
import java.io.Serializable;
import model.annotation.Id;

public class Customer implements Cloneable, Serializable

  { @Serial
  private static final long serialVersionUID = 1L;

  private static long customerCounter = 100;

  @Id
  private final String customerId;
  private String name;
  private String email;
  private String mobile;
  private Address address;

  public Customer(String name, String email, String mobile)
    { this.customerId = generateCustomerId();
    this.name = name;
    this.email = email;
    this.mobile = mobile;
  }

  public Customer(String name, Address address)
    { this(name, "", "");
    this.address = address;
  }
```

```
private static String generateCustomerId()
    { return "CUST" + (++customerCounter);
}

public String getCustomerId()
    { return customerId;
}

public String getName()
    { return name;
}

public String getEmail()
    { return email;
}

public String getMobile()
    { return mobile;
}

public Address getAddress()
    { return address;
}

public static class Address implements Serializable

    { @Serial
    private static final long serialVersionUID = 1L;

    private String line;
    private String city;
    private String pincode;

    public Address(String line, String city, String pincode)
        { this.line = line;
        this.city = city;
        this.pincode = pincode;
    }

    public String getLine()
        { return line;
    }

    public String getCity()
        { return city;
    }

    public String getPincode()
        { return pincode;
    }
}

@Override
public Customer clone() {

    try {
        return (Customer) super.clone();
    } catch (CloneNotSupportedException e)
        { return null;
    }
}
```

}

OUTPUT

```
./build.sh
java -jar build/minibank.jar
Built /Users/dhruvasavaliya/Desktop/Charusat/OOP_Practicals/D26IT107-OOP/MiniBank/build/minibank.jar
Loaded 3 saved account(s).
=====================================
Account Validation
=====================================
Invalid account: AC0004 | Validation Demo | -100.0
Validation errors:
  - balance must be > 0
Valid account: AC0001 | Dhruva | 8400.0
Validation errors: none

Accounts sorted by balance:
AC0003 | Aarya | 10000.0 | interest 7.0% per year | estimated yearly interest: 700.00
7. Run concurrency demo
8. Process transactions
9. Generate end-of-day report
10. View transaction history
11. Repository lookup demo
12. Exit
Choose an option: 11

Repository lookup for AC0001: AC0001 | Dhruva | 8400.0
Accounts stored in the repository:
AC0001 | Dhruva | 8400.0
AC0002 | Bhakti | 1500.0
AC0003 | Aarya | 10300.0
Customer repository lookup: Repository Demo
Average account balance: 6666.67
Main Menu
---------
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. List accounts
6. View statement
7. Run concurrency demo
8. Process transactions
9. Generate end-of-day report
10. View transaction history
11. Repository lookup demo
12. Exit
Choose an option: 12
Thank you for banking with MiniBank. Goodbye!
Bank session closed.
Saved 3 account(s) to data/accounts.dat.
```

PRACTICAL - 14 (A)

Aim: 1. Assemble and test: build a tiny layered app (a model, a service using an ArrayList, and a menu main), then write two or three JUnit @Test methods checking that adding increases the count and an invalid action throws — practice before testing MiniBank.

PRACTICAL - 14 (B)

Aim: 1. Create a class named BankService that uses Repository<Account>, the Validator and the exception classes to provide openAccount, deposit, withdraw, transfer, statement and listAccounts.

2. Connect the Practical 1 menu so each option calls the matching BankService method; load saved accounts on start-up (Practical 11) and save them on exit.

3. Use the TransactionProcessor (Practical 10) over the thread-safe store (Practical 12) to process a batch of transactions safely, and add the report (Practical 11) as a menu option.

4. Write a few JUnit tests using @Test: a deposit increases the balance; a withdrawal over the limit throws an exception; many concurrent deposits keep the balance correct.

5. (Optional) Add an @Audit annotation and a reflective logger that records which service method was called.

6. Build the final runnable JAR and complete the README.

FINAL OUTPUT

```
==========================================
Welcome to MiniBank – Charusat Branch
Your everyday banking, made simple
==========================================

Select an option to continue.
Main Menu
---------
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. List accounts
6. View statement
7. Process deposit batch
8. Generate end-of-day report
9. View transaction history
10. Exit
Choose an option: 1
Account type: 1. Savings  2. Current  3. Fixed Deposit
Choose account type: 1
Owner name: Riyansh
Email address: riya126@gmail.com
Mobile number: 9832561729
Opening balance (rupees): 4000
Customer created: CUST101
Account opened: AC0006 | Riyansh | 4000.0
Interest rate: 4.0% per year
Main Menu
---------
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. List accounts
6. View statement
7. Process deposit batch
8. Generate end-of-day report
9. View transaction history
10. Exit
Choose an option: 2
Account number: AC0002
Deposit amount: 5000
Deposit successful. New balance: 6310.0
Main Menu
---------
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. List accounts
6. View statement
7. Process deposit batch
8. Generate end-of-day report
9. View transaction history
10. Exit
Choose an option: 3
Account number: AC0003
Withdrawal amount: 3609
Transaction failed: Fixed deposit is locked; withdrawals are not allowed.
Main Menu
---------
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. List accounts
```

```
6. View statement
7. Process deposit batch
8. Generate end-of-day report
9. View transaction history
10. Exit
Choose an option: 5

Account list:
AC0003 | Aarya | 12000.0 | interest 7.0% per year | estimated yearly interest: 840.00
  Fixed deposit is locked; withdrawals are not allowed.
AC0001 | Dhruva | 9400.0 | interest 4.0% per year | estimated yearly interest: 376.00
AC0002 | Bhakti | 6310.0 | interest 0.0% per year | estimated yearly interest: 0.00
  Premium account
AC0005 | Aditi | 5000.0 | interest 4.0% per year | estimated yearly interest: 200.00
AC0006 | Riyansh | 4000.0 | interest 4.0% per year | estimated yearly interest: 160.00
Main Menu
---------
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. List accounts
6. View statement
7. Process deposit batch
8. Generate end-of-day report
9. View transaction history
10. Exit
Choose an option: 6
Account number: AC0001
Account Statement
Account Number: AC0001
Owner Name: Dhruva
Balance: 9400.0
Yearly interest estimate: 376.00
Main Menu
---------
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. List accounts
6. View statement
7. Process deposit batch
8. Generate end-of-day report
9. View transaction history
10. Exit
Choose an option: 7
Account number: AC0005
Number of deposits to process (1-1000): 6
Amount for each deposit (rupees): 200
Batch completed successfully: 6 deposits of ₹200.
Updated balance: ₹6,200.00 (was ₹5,000.00)
Main Menu
---------
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. List accounts
6. View statement
7. Process deposit batch
8. Generate end-of-day report
9. View transaction history
10. Exit
Choose an option: 8
MiniBank End-of-Day Report
Log files:
  transactions.log | 316 bytes
Total deposits: 26600 rupees
Total withdrawals: 5690 rupees
Net change: 20910 rupees
Unrecognized log lines: 0
```

```
Report saved to data/reports/end-of-day-report.txt.
Main Menu
---------
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. List accounts
6. View statement
7. Process deposit batch
8. Generate end-of-day report
9. View transaction history
10. Exit
Choose an option: 9

Transaction history:
DEPOSIT AC0002 3000
WITHDRAW AC0002 3400
DEPOSIT AC0001 3400
DEPOSIT AC0001 3000
DEPOSIT AC0005 5000
WITHDRAW AC0002 290
WITHDRAW AC0001 2000
DEPOSIT AC0003 2000
DEPOSIT AC0006 4000
DEPOSIT AC0002 5000
DEPOSIT AC0005 200
DEPOSIT AC0005 200
DEPOSIT AC0005 200
DEPOSIT AC0005 200
DEPOSIT AC0005 200
DEPOSIT AC0005 200
Main Menu
---------
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. List accounts
6. View statement
7. Process deposit batch
8. Generate end-of-day report
9. View transaction history
10. Exit
Choose an option: 10
Thank you for banking with MiniBank. Goodbye!
Your account data has been saved.
```